Vietnam National University Ho Chi Minh City
Ho Chi Minh City University of Technology
Faculty of Computer Science and Engineering

– INTERNSHIP 2 REPORT –

Cost-Aware Validation Scheduling and Selected Hints for LLM-Based Program Repair

Council: Computer Science and Engineering
Instructor: Dr. Le Xuan Bach, CSE-HCMUT
Student: Nguyen Thai Thanh Dat
Student ID: 2570387

Ho Chi Minh City, November 2026

Date: _____

Dr. Le Xuan Bach, CSE-HCMUT
Faculty of Computer Science and Engineering

Acknowledgements

I would like to thank the lecturers of the Faculty of Computer Science and Engineering, Ho Chi Minh City University of Technology (HCMUT), for the foundational and specialised teaching that made this work possible.

I am especially grateful to my instructor, Dr. Le Xuan Bach, CSE-HCMUT, for the guidance that shaped the scope of this work. Two pieces of that guidance did most of the work. The first was the insistence that a research question be stated together with the outcome that would *refute* it; applying that test removed several items from an earlier list that turned out to be names for groups of experiments rather than questions. The second was the instruction to measure the resource the system actually spends rather than the one the literature happens to report, which is why Chapter 5 reports CPU seconds and test executions side by side instead of either alone.

I also thank my colleagues for the discussions that sharpened the central distinction this work is built on: that a repair loop learns two different things on every iteration — which test is worth running, and which repair idea is already dead — and that both are usually discarded.

This document reports work in progress. The implementation and its unit tests are complete; the 100-task pilot is running at the time of writing, and Chapter 6 marks every number that depends on it. Any remaining errors are my own.

Abstract

A program repair tool built on a large language model (LLM) works in a loop: it asks the model for a patch, runs the tests to see whether the patch is wrong, and if it is, feeds the failure back and asks again, until a patch passes or the budget runs out. Two things make that loop expensive: running the tests is the step repeated most often and costing the most, and the model keeps proposing patches it has already tried, each still to be tested. Both have the same cause — the loop produces useful information on every iteration (which test rejected which patch, how long that test took, which repair ideas have already failed) and then throws almost all of it away.

This report describes CASTER, a repair loop that keeps that information in a small per-test record and spends it twice. **Cost-aware validation scheduling** combines how often a test has rejected patches with how long it takes to run, and picks the first test to run from the combination, so a bad patch is rejected as cheaply as possible. **Selected hints** replace the usual practice of pasting the whole conversation history into the prompt with a small, freshly chosen set of concrete facts pointing the model away from ideas already shown to fail. The two feed each other: the scheduler produces the failure information the hints are selected from, and better hints mean fewer wasted patches reaching validation. The whole mechanism is three numbers per test case, updated in constant time, with nothing trained and no extra test run or model call.

What was measured. The scheduling half was evaluated on a pilot of 97 Python tasks from ConDefects [1], three seeds each, a budget of twenty rounds. Against the test order the benchmark ships, the scheduler removed **60.8** % of validation CPU (95 % interval $[-73.3, -36.3]$) while repairing exactly the same bugs on exactly the same rounds. An ablation shows nearly all of that comes from keeping any memory at all: the published baseline FRTP [2], ranking tests by a plain count of past rejections, reaches -59.7 % on its own. The cost term adds a further 2.9 % over FRTP ($[-11.0, +4.9]$), a whole-session figure that averages two opposite regimes: a rejection count is informative after one observation, a cost estimate is a mean, so the scheduler is 2.1 % *more* expensive over rounds 1–5 and 10.6 % cheaper over rounds 11–15, with the gap still widening when the budget ends. The win concentrates in the third of tasks whose tests genuinely differ in cost, and is visible only because the study reports seconds: in test executions the comparison reverses sign.

The generation half is implemented and unit-tested but not yet run, and no number here is a corpus-scale result.

Contents

Acknowledgements **iv**

Abstract **v**

1 Introduction **1**
1.1 The Repair Loop . . . 1
1.2 Two Problems with Current Practice . . . 1
1.3 The Information the Loop Discards . . . 2
1.4 What Existing Solutions Miss . . . 2
1.5 This Work: Caster . . . 2
1.6 Contributions . . . 3
1.7 What the Pilot Found . . . 4
1.8 Structure of This Report . . . 4

2 Related Works **5**
2.1 Reducing Patch Validation Cost . . . 5
2.2 Steering Generation in LLM-Based Repair . . . 7
2.3 Context Selection for LLM-Based Repair . . . 7
2.4 Summary: Where This Work Sits . . . 8

3 Background **9**
3.1 Iterative LLM-Based Repair . . . 9
3.2 Validation Cost and Test Ordering . . . 10
3.3 Repair Hypotheses and History in the Prompt . . . 13
3.4 Benchmark Background . . . 14

4 Methodology **15**
4.1 Overview . . . 15
4.2 The Repair History Store . . . 17
4.3 Cost-Aware Validation Scheduling . . . 18
4.4 Selected Hints . . . 19

5 Experiments **24**
5.1 Research Questions . . . 24
5.2 Benchmark and Corpus . . . 25
5.3 Configurations . . . 26
5.4 Metrics . . . 28
5.5 How the Numbers Are Produced . . . 28

6 Results **30**
6.1 What the Pilot Measured 30
6.2 RQ1a — Is Scheduling the Oracle Worth Doing? 32
6.3 RQ1b — Against the Published Prioritisers 34
6.4 RQ2 — Which Term Carries the Effect 37
6.5 RQ3 and RQ4 — Not Yet Run 38
6.6 Summary of Findings 39

7 Discussion **40**
7.1 When the Method Does Not Help 40
7.2 Reporting Seconds Rather Than Executions 41
7.3 Composing with the Other Two Families, and What Each Costs to Adopt 42
7.4 What the Pilot Can and Cannot Settle 43

8 Threats to Validity **44**
8.1 Internal Validity 44
8.2 External Validity 44
8.3 Construct Validity 45
8.4 Conclusion Validity 46

9 Conclusion and Future Work **47**
9.1 Conclusion 47
9.2 Future Work 48

References **49**

Chapter 1

Introduction

1.1 The Repair Loop

A *bug* is a place where a program does the wrong thing. A *test case* is one concrete check: an input, and the output the program is supposed to produce for it. A *test suite* is a collection of such checks, and a *patch* is an edit to the program that is meant to make the failing checks pass. *Automated program repair* (APR) is the task of producing such a patch without a human writing it [3].

In APR based on large language models (LLMs), one design dominates: an iterative *generate–validate–refine* loop. Given a buggy program and its failing tests, the LLM generates a candidate patch for the suspicious code. The patch is then *validated* by running the test suite against it. If validation fails, the resulting feedback — the failing test cases, the error message, or the stack trace — is fed back to the model so that it can refine the patch, and the loop repeats. It stops when a patch passes all tests, which makes it *plausible*, or when the iteration budget is exhausted. Validation sits *inside* the cycle, so its cost is paid once per candidate patch rather than once per bug — which is the whole reason it dominates.

1.2 Two Problems with Current Practice

Current practice reveals two issues with this loop.

Validation is the most repeated and the most expensive step. Each candidate patch must be built and executed against the test suite before its correctness can be judged, and this check is repeated for every candidate the LLM produces. Lin et al. [4] analysed 7,745 agent traces from SWE-bench leaderboard submissions and found that every agent and model examined relies on code execution, averaging 8.8 test runs per task, with frequency varying widely from 2 to 19 runs per task across agents and models. Each of these runs is costly: RepairAgent [5] spends 99 % of its total repair time on tool executions, mostly running tests, with a median repair time of about 920 seconds per bug. Validation therefore dominates the end-to-end cost of LLM-based repair, and every candidate that reaches this step adds directly to that cost.

LLMs regenerate patches they have already tried. When candidates are sampled independently, the LLM is unaware of its previously generated patches and may

produce many repeated or similar candidates [6]. Exposing this history does not eliminate the problem: ChatRepair [6] keeps earlier incorrect patches and their test-failure information in the conversation, yet Chen et al. [7] found that duplicate patches still constituted 65 % of all patches it generated during plausible patch generation on Defects4J bugs. Feedback-driven refinement does not fully resolve the problem either: when the underlying repair hypothesis is flawed, refinement tends to produce only superficial variants of the same incorrect idea [8]. Since every such candidate must still be validated, this redundancy directly inflates the cost of the already expensive validation step.

1.3 The Information the Loop Discards

Both issues stem from the same root: the repair loop produces valuable information but makes little use of it. Every validation reveals which tests reject which candidates and at what cost, and every generation reveals which repair hypotheses have already been explored. This history offers two levers. It can **schedule validation**, so that each candidate is judged as cheaply as possible, and it can **steer generation**, so that the LLM is directed away from directions that have already failed.

1.4 What Existing Solutions Miss

Prior work reduces validation cost by making each test run cheaper [9, 10], by skipping execution with learned verifiers [11], or by scheduling which tests to run [2, 12]. Our work targets scheduling, which is orthogonal to execution acceleration. Existing schedulers, however, optimise the *number* of test executions rather than their *cost*. FRTP [2], for instance, first runs the tests that failed earlier candidates, regardless of how long each one takes, even though long-running tests are what make validation expensive.

On the generation side, prior work explores the repair space through tree search [13], retains earlier attempts in memory [6, 14], reasons explicitly about bug hypotheses [15, 16], or transfers experience across bugs [17]. Yet *how* history is presented matters as much as *whether* it is presented. As noted above, ChatRepair still produces mostly duplicate patches despite seeing its own history [7]. Parasaram et al. [18] further show that adding more facts to the prompt can hurt repair, but their selection is made once, over static facts, before generation. How the evolving history of a single repair session should be condensed, at each iteration, into guidance toward untried repair directions remains open.

1.5 This Work: Caster

To fill these gaps, this work proposes Caster, an LLM-based repair approach that turns the repair history into two complementary mechanisms.

Cost-aware validation scheduling combines the failure history of each test with its execution cost to decide which tests to run first, so that unpromising patches are rejected with minimal execution time.

Selected hints replace the accumulation of raw history in the prompt with a small, iteration-specific set of hints, distilled from previously tried patches and their failures, that steer the LLM away from repair hypotheses already shown to fail.

The two mechanisms reinforce each other: scheduling produces the failure information from which hints are selected, and better-steered generation reduces the number of redundant candidates that reach validation in the first place. Because CASTER operates at the level of scheduling and prompting, it can be combined with execution-acceleration frameworks [9, 10] and execution-free verifiers [11].

1.6 Contributions

1. **A per-test repair history store.** One record per test case, updated as the loop runs, holding how many times the test has been run, how many of those runs rejected a patch, and how much CPU time those runs took (Section 4.2). The same record serves both mechanisms, which is why they ship together rather than as two independent features.

2. **Cost-aware validation scheduling.** A scheduling rule that ranks tests by an estimated rejection rate divided by an estimated cost, so that the loop prefers tests that are likely to reject a patch *and* cheap to run (Section 4.3). Existing repair-loop schedulers rank on a count of past rejections only.

3. **Selected hints.** A prompt contract that, at every iteration, selects a small number of concrete facts from the history and renders them as requirements the next patch must satisfy, rather than appending the full transcript (Section 4.4).

4. **A mechanism cheap enough to adopt.** Both mechanisms run on one record of three numbers per test case, updated in constant time (Section 4.1.2). Nothing is trained, no profiling pass is needed, and neither adds a program execution or a model call — the scheduler reorders runs the loop was already making, and hint selection only reads what validation has already recorded. This matters for comparison as well as deployment: the other two families of validation-cost reduction each require a language-runtime integration or a trained verifier before they return anything (Section 7.3).

5. **An evaluation protocol that reports the resource actually spent.** CPU seconds as the primary unit with test executions reported alongside, paired per task, with interval estimates from a bootstrap over tasks (Chapter 5). The two units can disagree, and on this pilot they did: against the published baseline the proposed scheduler runs 13 % more test executions and spends 3 % less CPU, so a result reported in executions alone would have reversed the conclusion.

1.7 What the Pilot Found

The scheduling half has been measured on a pilot of 97 ConDefects Python tasks; the generation half has not been run. Two results are worth stating up front, because the second is narrower than the contributions above might suggest.

Against the test order the benchmark ships, the proposed scheduler removes 60.8 % of validation CPU and repairs exactly the same bugs on exactly the same rounds. But **almost all of that comes from keeping any memory at all**: FRTP, published in 2013 and ranking on a plain count of past rejections, reaches 59.7 % on its own. The cost term adds a further 2.9 % with an interval that includes zero — although that whole-session figure averages two opposite regimes, since a cost estimate needs more observations than a rejection count does: the scheduler is 2.1 % more expensive over the first five rounds and 10.6 % cheaper over rounds 11–15, and the gap is still widening when the budget ends.

1.8 Structure of This Report

Chapter 2 surveys related work in three groups: reducing patch validation cost, steering generation, and context selection. Chapter 3 builds the necessary background from first principles. Chapter 4 describes CASTER. Chapter 5 states the four research questions and the protocol for the pilot, and Chapter 6 reports the measured results for the scheduler and the layout of the generation-side tables that have not been run. Chapters 7–9 discuss when the approach does and does not help, state the threats to validity, and conclude.

Chapter 2

Related Works

Work related to this report falls into three groups. The first reduces the cost of checking a candidate patch. The second helps the model explore the repair space instead of repeating itself. The third asks what should go into the prompt at all. CASTER touches all three: it schedules validation, it steers generation, and it does so by selecting a small amount of context rather than accumulating all of it.

2.1 Reducing Patch Validation Cost

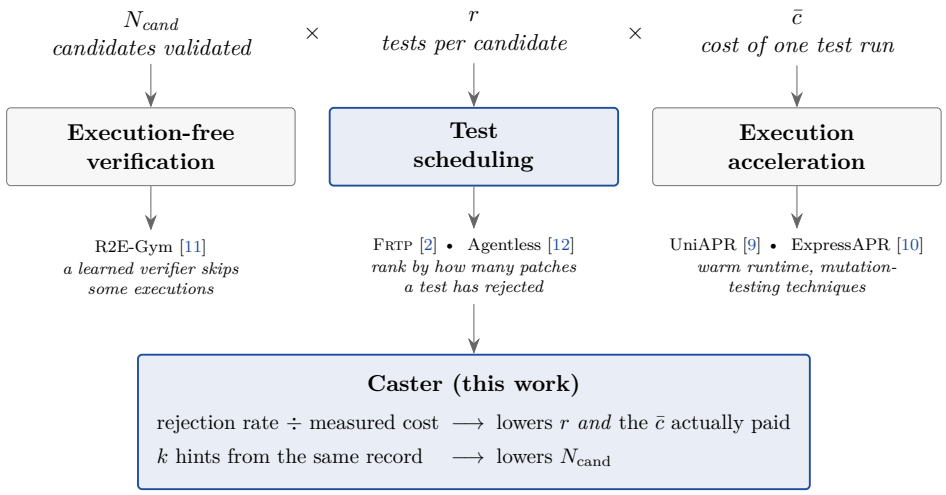
Patch validation has long been recognised as a bottleneck of test-based repair, and prior work reduces its cost along three largely complementary axes. The three are easiest to tell apart by asking which part of the bill each one attacks. Writing the total validation cost of a repair session as a product of three factors,

$$\text{validation cost} \approx \underbrace{N_{\text{cand}}}_{\substack{\text{candidates that}\\ \text{reach validation}}} \times \underbrace{r}_{\substack{\text{tests run}\\ \text{per candidate}}} \times \underbrace{\bar{c}}_{\substack{\text{cost of one}\\ \text{test run}}}, \tag{2.1}$$

each family reduces a different factor, and none of them reduces another family's. Figure 2.1 places the surveyed systems on that decomposition and marks where this work sits.

Execution acceleration makes each test run cheaper. Because patch validation repeatedly runs the same test suite on slightly different program variants, it is closely related to mutation analysis, and several works adapt acceleration techniques from that field. UNIAPR [9] validates patches on the fly within a single JVM, reloading only the patched classes through the HotSwap mechanism and resetting JVM state between patches to avoid interference. EXPRESSAPR [10] identifies five classes of mutation-testing acceleration techniques suitable for patch validation and integrates them into a general-purpose validator for Java, accelerating validation by 137.1× over plain validation and 8.8× over the prior state of the art. These techniques reduce the cost of each execution but leave unchanged *which* tests are executed and *in which order*, and they are tied to a specific language runtime.

Execution-free verification avoids running tests for some candidates by assessing them with a learned model. In repository-level repair, R2E-GYM [11] compares



The three families multiply rather than compete: a deployment can run an accelerated executor, a cost-aware schedule on top of it, and a verifier in front of both.

Figure 2.1: The three directions in reducing patch validation cost, placed on the factors of Equation (2.1). This work sits in test scheduling, and is the only entry we found there whose ranking reads the cost of a test as well as its record of rejecting patches.

execution-based and execution-free verifiers and finds their strengths complementary. Test-based verifiers often fail to distinguish among candidates, while execution-free verifiers are biased and tend to rely on stylistic features. Combining the two yields substantially higher performance than either alone. Such verifiers are therefore best seen as a complement to test execution rather than a replacement for it.

Test scheduling decides which tests to run and in which order. FRTP [2] records the tests that failed earlier candidate patches during the repair process and executes them first, so that invalid patches are rejected after fewer test executions. Because this information comes from the repair process itself, FRTP incurs only trivial overhead. Its implementation, TRPAUTOREPAIR, performed at least as well as GENPROG in success rate while significantly improving repair efficiency, on 15 of 16 real-world bugs. In LLM-based repair, AGENTLESS [12] first records which existing tests pass on the original program and asks the LLM to select those relevant to the edited code. It then rejects any candidate that causes a selected test to fail.

These schedulers optimise the *number* of test executions rather than their *cost*. In regression testing, by contrast, it has long been recognised that test costs vary and that prioritisation should account for them [19]. In patch validation this distinction matters: a test that rejects many candidates but takes minutes to run may be a worse first choice than a slightly less discriminating test that finishes in milliseconds.

Our approach departs from these schedulers by scheduling tests using both their failure history within the repair session and their execution cost. Because it operates purely at the scheduling level, it can be combined with execution acceleration [9, 10] and execution-free verification [11].

2.2 Steering Generation in LLM-Based Repair

A second line of work aims to help LLMs explore the repair space more effectively instead of repeatedly sampling similar candidates.

Search and in-loop memory. SWE-Search [13] organises repair attempts into a Monte Carlo tree search with an LLM value function, improving on SWE-bench by 23 % relative; but every additional branch produces candidates that must be validated, which increases the cost this work reduces. ChatRepair [6] instead keeps earlier incorrect patches and their test failures in the conversation, so the model can avoid repeating them. Chen et al. [7] found that duplicate patches still made up 65 % of what it generated, and that plain independent resampling repaired *more* bugs at an equal budget. When the underlying repair hypothesis is flawed, feedback-driven refinement tends to produce only superficial variants of the same idea [8]. Outside repair, Reflexion [14] stores verbal reflections on failed trials in an episodic memory. In all of these the history is accumulated or summarised *as it arrives*, rather than selected according to what the next iteration needs.

Hypotheses and transferred experience. A second group makes the model reason explicitly about the cause. AutoSD [15] prompts for hypotheses together with debugger experiments that test them; ThinkRepair [16] builds a pool of chains of thought from validated fixes and selects few-shot examples from it. Both improve the quality of individual hypotheses without tracking which hypotheses this session has already refuted. SWE-Exp [17] goes further and transfers distilled experience across bugs — but that experience comes from *other* bugs, whereas the history most informative for avoiding a repeat is the one produced on the current one.

In contrast, Caster does not accumulate or summarise the session history. At each iteration it selects a small set of hints from it, ranked by the per-test rejection statistics its scheduler already maintains, so the model is directed by the tests that most often refute candidates and away from the hypotheses they have refuted — and, unlike search, without generating extra candidates.

2.3 Context Selection for LLM-Based Repair

The effectiveness of LLM-based repair depends heavily on what information is included in the prompt. Parasaram et al. [18] systematically studied the bug-related facts that can be provided to an LLM, such as stack traces, test code, and repository context. They found that while each type of fact helps fix some bugs, repair effectiveness is non-monotonic in the number of facts included, so overly rich prompts degrade performance, and no single fact set is optimal across bugs. Their bug-specific fact selector repaired 88 of 157 bugs, 17 % above the best configuration. This selection, however, operates on *static* facts about the bug and is performed once, before generation begins.

CASTER applies the same principle — that less but well-chosen context is better — to the *dynamic* history of an iterative repair loop. It reselects a small set of hints at every iteration as new patches are tried and new failures are observed.

2.4 Summary: Where This Work Sits

Figure 2.1 separated the surveyed work by *which part of the bill* it reduces. This section asks a narrower question: of the design decisions CASTER makes, which ones has someone already made?

Three dimensions distinguish it. **D1:** does the system decide *which test to run first*? **D2:** does the *measured cost* of a test enter that decision? **D3:** does the system *choose* what evidence enters the prompt, rather than including all of it or none?

Each dimension is satisfied by published work on its own; what we found no occupant for is the combination. Blank cells in Table 2.1 mean we are not aware of a system in that position, not that none exists, and D2 is only meaningful alongside D1, since cost can enter a ranking only if there is a ranking.

Only systems holding at least one of D1–D3 appear; those that reduce a different factor of the bill, or change the search rather than the evidence, are placed in Figure 2.1 instead, because scoring them here would produce a row of dashes and compare nothing.

Table 2.1: The systems that occupy at least one of D1–D3, and what still separates each from this work. ✓ is full, ○ is partial, blank is absent.

System	D1	D2	D3	What is still different here
Additional greedy [20]	✓			Ranks by coverage of the program, a static property, not by anything the repair loop observes
FRTP [2]	✓			Ranks on a *count* of rejections: no denominator, so it cannot tell an unrun test from one that never rejects; and no cost term
AGENTLESS [12]	✓			The model picks which tests look relevant; nothing records what actually rejected what, and nothing is timed
Cost-cognizant TCP [19]	✓	✓		Regression testing, not repair. Costs are supplied before the run rather than measured inside it, and no evidence is returned to a generator
Fact selection [18]			✓	Selects *static* facts about the bug, once, before generation begins. Says nothing about validation, and does not reselect as the session produces new failures
CHATREPAIR [6], REFLEXION [14]			○	History reaches the prompt, but it is accumulated rather than selected, so it grows with the session and dilutes the few facts that matter
Caster (this work)	✓	✓	✓	One per-test record drives both the schedule and the prompt

Three gaps follow, and together they are what this work adds. Repair-loop schedulers ignore cost, and the one line of work that puts cost in the ranking [19] assumes it is known in advance. A count of rejections discards the evidence of *not* rejecting, since a test run ten times that rejected nothing and a test never run both score zero. And session history is accumulated rather than selected — fact selection [18] does select, but once, over facts fixed before the first candidate is drawn.

Chapter 3

Background

This chapter builds up the objects the rest of the report uses, starting from plain definitions. Section 3.1 defines the repair loop precisely. Section 3.2 explains what validation costs, why the order in which tests run changes that cost, and why counting test runs is not the same as measuring time. Section 3.3 explains what a repair hypothesis is and why putting the whole history in the prompt does not stop the model repeating one.

3.1 Iterative LLM-Based Repair

3.1.1 The Objects

Table 3.1 lists every term this report uses in a technical sense. Each is defined once here and used consistently afterwards.

Table 3.1: Terms used in this report.

Term	Meaning in this report
Faulty program s_0	The program that contains the bug.
Test case x	One input together with the output the program is expected to produce for it.
Test pool P	The finite set of test cases available for one task.
Reject (refute)	A test case *rejects* a program when its output on that input differs from the expected one. One rejecting test proves the program wrong.
Candidate patch s	A modified program produced by the LLM in one round.
Round	One pass of generate $\rightarrow$ validate $\rightarrow$ refine. A *session* is all rounds spent on one task, up to the budget B.
Scan	One validation: running a candidate against test cases from P until one rejects it, or until all have passed.
Plausible patch	A candidate no test case in P rejects — "plausible" rather than "correct" because passing a finite pool is weaker than being right [21].
Test-run cost $c(x)$	The CPU time consumed by running one candidate on one test case x.
Repair hypothesis	The idea behind a patch: what the model believes is wrong and how it intends to fix it. Many patches can express one hypothesis.

3.1.2 The Loop as a Formal Object

Definition 3.1 (Repair session)**.** A repair session with budget B on a faulty program s_0 and a test pool P is the following sequence, for $t = 1, 2, \ldots, B$:

1. The LLM draws a candidate patch $s_t \sim \pi(\cdot \mid \mathcal{H}_{t-1})$, where $\mathcal{H}_{t-1}$ is whatever the loop has chosen to put in the prompt from rounds $1, \ldots, t-1$.
2. The *validator* runs s_t against test cases from P in some order σ_t, stopping at the first test case that rejects s_t.
3. If no test case rejects s_t, the session *accepts* s_t and stops. Otherwise the rejecting test case, together with whatever else the loop chooses to carry, becomes part of $\mathcal{H}_t$, and the next round begins.

Two choices are left open by Definition 3.1, and they are exactly the two levers of this work. The order σ_t is free, and nothing in the definition says it must be the order the benchmark happens to ship. The contents of $\mathcal{H}_t$ are free, and nothing says they must be the full transcript.

3.1.3 The Two Bills

A repair session spends two different resources, and keeping them apart matters because optimising one can increase the other.

- **The generation bill**, paid in model tokens and therefore in money or in GPU time. It grows with the number of rounds and with the size of the prompt.
- **The validation bill**, paid in CPU seconds spent executing candidate programs. It grows with the number of rounds, with how many test cases each scan runs, and with how expensive those particular test cases are.

This report targets the validation bill with its scheduler and the generation bill with its hints, and reports both.

3.2 Validation Cost and Test Ordering

3.2.1 What a Session Costs

Write $c(x)$ for the CPU cost of running one candidate on test case x, and σ_t for the order the validator uses in round t, so that $\sigma_t(j)$ is the j-th test case it runs. The validation cost of a session is

$$C \;=\; \sum_{t=1}^{T} \sum_{j=1}^{r_t} c(\sigma_t(j))\,, \qquad r_t \;=\; \min\{\, j \;:\; \sigma_t(j) \text{ rejects } s_t \,\}, \tag{3.1}$$

where T is the number of rounds actually run and $r_t = |P|$ when nothing rejects s_t. Two observations about Equation (3.1) drive the whole design.

First, σ_t is the only term the validator controls. The candidate comes from the model, the pool comes from the benchmark, and the per-test costs are properties of the programs; the order is a free choice made on every round.

Second, minimising the *number* of test runs $\sum_t r_t$ is the same objective as minimising C **only if** c **is constant**. When test costs differ, a schedule can run more tests and still finish sooner, or run fewer tests and take longer. The two quantities can therefore rank two schedules in opposite orders.

3.2.2 Why Executions and Seconds Are Not Interchangeable

The test-prioritisation literature usually reports the number of test executions rather than elapsed time, and does so for a defensible reason. The largest study of patch validation to date measures executions precisely because elapsed time is unstable across machines and configurations, where an execution count reproduces [22]. That choice is sound when executions are interchangeable — when running any one test costs about as much as running any other.

It stops being sound when they are not. In the setting studied here, a test case is one whole-program execution on one input, and inputs within a single task's pool differ by orders of magnitude in size and in the amount of work they provoke. A deployment pays for seconds, not for executions, so a result reported only in executions does not tell the deployment what it will pay. This is why Chapter 5 makes CPU seconds the primary unit and reports executions alongside rather than instead.

3.2.3 Order Changes the Cost but Not the Answer

Definition 3.2 (Soundness of a schedule)**.** A candidate is accepted only after surviving *every* usable test case in P. Acceptance is therefore independent of the order σ_t: no ordering can accept a patch that another ordering rejects, and no ordering can reject a patch that another ordering accepts.

Definition 3.2 is what makes a pure cost claim possible. If two schedules repaired different numbers of bugs, the difference would be an implementation bug rather than a finding, so the experiment asserts the equality rather than reporting it (Section 5.4).

3.2.4 Test-Case Prioritisation

The problem of choosing σ_t is an instance of *test-case prioritisation*, studied for decades in regression testing. Rothermel et al. [20] formalised it as the search for a permutation of the test pool that maximises how quickly faults are detected, and introduced the two greedy families still in use:

- **Total** strategies rank a test case by a static property, usually how many lines of the program it exercises.
- **Additional** strategies rank by *marginal* gain: how many lines it exercises that nothing scheduled so far has exercised, re-ranking after every pick. This family, written RTP_{add}, is the stronger of the two in most empirical comparisons and is used in this report as a classical baseline.

Elbaum et al. [19] observed that the standard metric for prioritisation assumes uniform test costs, and proposed a cost-cognizant replacement. That paper is the ancestor of the cost term in this work. What it did not have is an *online* cost estimate: its costs are supplied before the run, whereas a repair loop can measure them as it goes, for free, because it is running the tests anyway.

3.2.5 What Frtp Records and What It Discards

FRTP [2] is the direct baseline for the scheduling half of this work. It ranks each test case by its *failing count*: the number of candidate patches that test case has already rejected during this repair session. The information is free, because the repair loop produces it as a by-product, which is why FRTP's overhead is negligible.

Writing $k(x)$ for that count, FRTP schedules in decreasing k. Two things are missing from k, and naming them precisely is what produces this work's scheduling rule.

Figure 3.1 makes both gaps concrete on four tests.

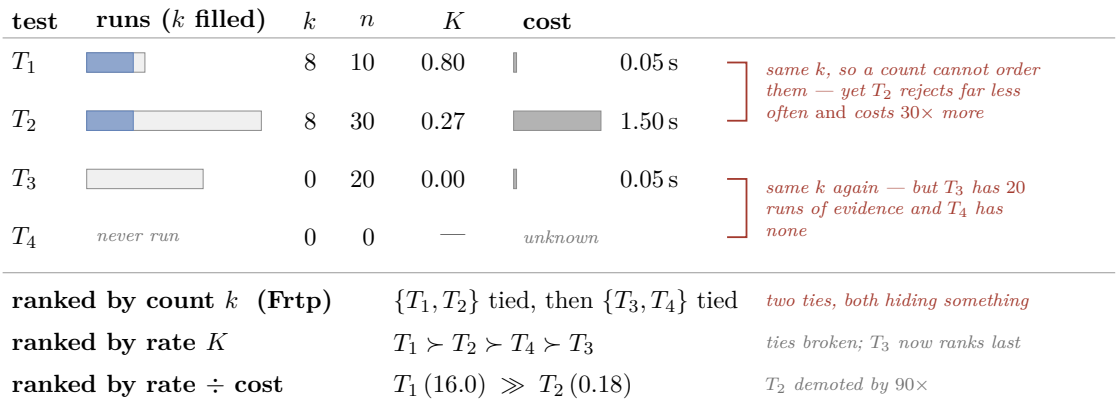


Figure 3.1: The two things a rejection count leaves out, on four tests. T_1 and T_2 have rejected the same number of patches, so FRTP cannot order them — yet T_1 rejects four candidates in five and T_2 roughly one in four, and T_2 costs thirty times as much to run. T_3 and T_4 also tie at zero, although T_3 has twenty runs of evidence that it never rejects and T_4 has none. Reading the denominator breaks both ties; dividing by cost then demotes the expensive test.

The denominator is missing. Let $n(x)$ be the number of times test case x has been *run*. A count $k(x)$ without $n(x)$ cannot distinguish a test case that was run ten times and rejected nothing from a test case that was never run at all: both have $k = 0$. The ratio $k(x)/n(x)$ is the first quantity that reads the evidence of *not* rejecting, and it is also the first that lets a test case whose early record was luck decay back towards the average.

The cost is missing. Nothing in k knows that one test runs in milliseconds and another takes seconds. Under a uniform-cost assumption that is harmless. When costs are spread out it is not, and it is worse than neutral if expensive tests also tend to

be more discriminating — in that case ranking on rejections alone actively steers the schedule towards the expensive end of the pool.

3.3 Repair Hypotheses and History in the Prompt

3.3.1 Patches Versus Hypotheses

A *repair hypothesis* is the idea behind a patch: what the model believes is wrong and how it intends to fix it — "the loop bound is off by one", "integer division should be float division". One hypothesis can be written as many different patches, differing in variable names, statement order or whitespace.

The distinction matters for measurement. A *duplicate* patch, in the sense used by Chen et al. [7], is a candidate whose normalised text has already been generated this session. Two candidates that are textually different but express the same hypothesis are not duplicates by that definition, yet they are just as wasteful: both must be validated and both fail for the same reason. The duplicate rate is therefore a *lower bound* on the redundancy a repair loop produces, which is the sense in which the reported 65 % for ChatRepair is conservative.

3.3.2 Why Showing the History Is Not Enough

The natural fix for redundancy is to show the model what it has already tried. Three published observations say this is necessary but not sufficient. A transcript does not prevent repetition: ChatRepair keeps earlier incorrect patches and their failures in the conversation, and Chen et al. [7] still measured 65 % of its patches to be duplicates. Refinement preserves a bad hypothesis: when the underlying idea is wrong, feedback-driven refinement produces superficial variants of it rather than a new idea [8], and the loop pays for a validation that was predictable in advance. And more context can hurt: Parasaram et al. [18] found repair effectiveness to be *non-monotonic* in the number of facts in the prompt, so past a point, adding true and relevant information makes repair worse — and a transcript grows monotonically, so a long session necessarily walks past that point.

Taken together, the problem is not the *absence* of history but its *form*. A growing transcript is the worst of both: it costs tokens every round, it dilutes the few facts that matter, and it still leaves the model free to re-propose.

3.3.3 What a Hint Has to Be

Two properties separate a useful hint from a transcript line.

It must be an observation, not a category. A failure class — "the off-by-one family" — can only be phrased as a prohibition: *do not do that again*. A concrete

test case can be phrased as a positive requirement: *on this input, produce exactly this output*. A requirement tells the model what success looks like; a prohibition removes one option and leaves it to guess the rest. This is also why the scheduler and the hint contract are one system here: the scheduler's unit of memory is the test case, and the test case is precisely the object that can be stated as a requirement.

It must be selected, not accumulated. Because effectiveness is non-monotonic in prompt content, the right number of hints is small and roughly constant. Which hints those are changes from round to round, because the history changes — so selection must happen at every iteration, which is what separates this from selecting static facts once before generation begins.

3.4 Benchmark Background

ConDefects [1] collects real faulty submissions from AtCoder programming contests, together with the author's later accepted version and the contest's own test data: 1,254 Java and 1,625 Python faulty programs. It was built to address data leakage in LLM evaluation, since its faults post-date the training cut-offs of the models commonly benchmarked on Defects4J [23].

Four of its properties matter here. Every test input ships with its expected output, so a rejection is a concrete input/output pair rather than a pass/fail bit — which is what lets a hint be stated as a positive requirement. A test is one whole-program execution, so there is no call graph and nothing to *select away*; prioritisation is the only scheduling lever, which is the structural reason the "selection beats prioritisation" finding of Lou et al. [22] does not transfer here. Contest setters ship small public samples alongside large adversarial inputs, so a single task's pool spans a wide range of execution costs — the spread a cost-aware schedule needs. And a submission carries no problem statement, so every prompt must supply worked examples, identically in every configuration, or the comparison measures prompt content rather than policy.

Chapter 4

Methodology

4.1 Overview

CASTER is an iterative repair loop with one extra component: a small record kept for each test case, updated every time that test case is run. Everything else follows from what is in that record.

The record holds three numbers per test case: how many times the test has been run in this session, how many of those runs rejected a candidate, and how much CPU time those runs consumed. From those three numbers the loop derives two estimates — how likely the test is to reject the next candidate, and how long it takes — and spends them in two places. The **scheduler** uses both to decide which test to run first. The **hint selector** uses the first, together with how readable the test's input is, to decide which facts go into the next prompt.

Figure 4.1 shows one round.

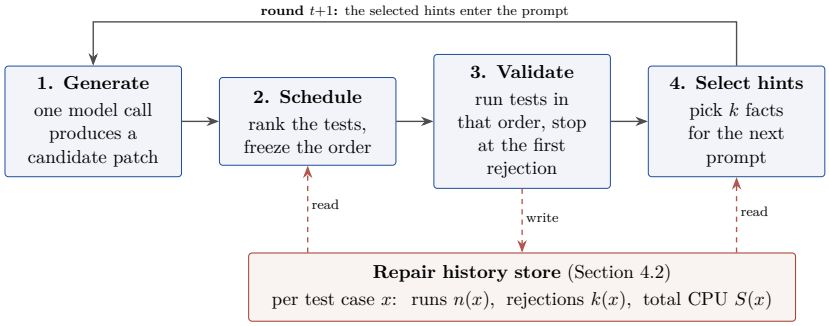


Figure 4.1: One round of the CASTER loop. The repair history store is written by validation and read by both the scheduler and the hint selector, which is why the two mechanisms are one system rather than two independent features.

4.1.1 Requirements the Design Must Satisfy

Five rules are binding on the implementation, because breaking any of them produces a plausible-looking but wrong number rather than a visible error. The store holds (n, k, S) per test case, and the order is derived from it **once per round** and then **frozen** — re-ranking mid-round would make the order depend on results from the same scan, which is not reproducible from the store alone. The scan **stops at the first rejection**, so only tests actually run update n and S, and only the rejecting test

updates k. A candidate is accepted only after surviving **every** usable test, so acceptance is independent of the order (Definition 3.2). And a hint is rendered as a **positive requirement** over a concrete input/output pair, never as a named prohibition; the prompt never names a test whose content it does not show.

Two measurement rules accompany them. Cost is read as a `getrusage (RUSAGE_CHILDREN)` delta — CPU time, not wall clock — because wall clock on a shared machine is contaminated by sibling processes and would make the learned cost estimate a function of machine load rather than of the test. And because `getrusage` accounting is per process, parallelism must be process-per-worker rather than threaded, or one worker's CPU is attributed to another.

4.1.2 What the Mechanism Costs

A method that reduces one cost is only worth having if it does not quietly introduce another. It is therefore worth saying precisely what Caster adds, because the answer is close to nothing: **three numbers per test case, updated in constant time, with nothing trained and nothing executed that the loop was not already going to execute.**

Per task, the store is $O(|P|)$ memory — three numbers per test, a few hundred bytes for a pool of forty. An update is two increments and an addition; a round is one sort of at most $|P|$ items, built lazily and abandoned at the first rejection. There are **no** extra program executions (the scheduler reorders runs the loop was already making), **no** extra model calls (hint selection only reads the store), **no** growth of the prompt with session length (at most k hints in any round, not $O(B)$ transcript lines), **no** training and **no** offline profiling pass — test costs are measured from the runs the loop pays for anyway.

Two consequences follow. **The information is free**: the scheduler's inputs — which test rejected which candidate, and how long that took — are by-products of work the loop performs regardless, which is the property that made Frtp [2] cheap enough to adopt. And **the component is small and portable**: the store is statistics over test-case identifiers, with no dependency beyond the standard library, no parsing of the program and no knowledge of the language, which is why it unit-tests without a benchmark, a sandbox or a model, and why any loop that can run a test and time it can keep this record.

One caveat, so the claim is not overstated. The *experiment* in Chapter 5 does run an extra full-scan `audit` pass over every patch and every test. That pass exists so the kill matrix and the cost table are established independently of which configuration ran first, which is a requirement of *comparing* schedules fairly. It is not part of the method, and a deployment running Caster does not perform it.

4.2 The Repair History Store

4.2.1 What Is Stored

For every test case x in the pool, the store holds three counters:

$n(x)$	how many times x has been run in this session
$k(x)$	how many of those runs rejected the candidate
$S(x)$	the total CPU seconds those runs consumed

Nothing else is stored, and in particular nothing about the program under test. The store is therefore pure statistics over test-case names: it can be built, inspected and unit-tested without a benchmark, a sandbox or a model, which is what makes the scheduler testable in isolation.

4.2.2 The Two Derived Estimates

The rejection rate. Treat "does x reject the next candidate?" as a coin flip with unknown bias, and estimate that bias from $k(x)$ successes in $n(x)$ trials. Using the standard Beta conjugate prior, the posterior mean is

$$K(x) \;=\; \frac{k(x)+\alpha}{n(x)+\alpha+\beta}. \tag{4.1}$$

The prior is doing real work here, because at the start of a session every test has $n = 0$ and the prior is the *only* thing ranking them.

The default is $\mathrm{Beta}(\frac{1}{2}, 2)$, whose mean is 0.2 — above the measured base rate on this corpus and comfortably below one half. The obvious alternative, Jeffreys' $\mathrm{Beta}(\frac{1}{2}, \frac{1}{2})$ [24], places a never-run test at $K = 0.5$, which outranks a test with a real record of 2 rejections in 10 runs (0.227). A rule that always prefers the unmeasured test to the proven one ends up running the whole pool, so under Jeffreys the scheduler stops scheduling. Trying new tests is the job of the exploration term below, not a side effect of the prior. Both priors ship, and the choice is part of the configuration key.

The cost estimate. The running mean cost of a test is $\hat{m}(x) = S(x)/n(x)$. A test that has never been run has no mean, and the cold-start value chosen for it decides how aggressively the loop tries new tests. Entering it at zero would make an unrun test infinitely attractive; entering it at the pool average would make the first round's order essentially arbitrary. Caster enters an unrun test at the *maximum* of the observed per-test means, which is pessimistic: an unmeasured test must earn its place through its rejection rate rather than through an optimistic guess about its cost. Two alternative cold starts, the median and the 75th percentile, are kept as sensitivity configurations (Section 5.3).

4.2.3 Update Rules

After each scan, the store is updated as follows. Every test that was actually run has its n incremented and its measured CPU added to S. The one test that rejected the candidate, if any, has its k incremented. Tests in the unrun tail of the order are untouched, because they cost nothing and revealed nothing.

One accounting rule is worth stating, because violating it silently corrupts the cost estimate. A test that *times out* is excluded from the cost model rather than entered at the timeout value. A timeout is weaker evidence than a wrong answer — a correct but slow program times out too — and entering timeouts at the clock value would let a handful of them dominate $\hat{m}$ for the whole pool.

4.3 Cost-Aware Validation Scheduling

4.3.1 The Ranking Rule

The scheduler ranks tests by

$$\text{score}(x) \;=\; \frac{K(x)}{\hat{m}(x)} \tag{4.2}$$

and runs them in decreasing score until one of them rejects the candidate.

The intuition is a *rate of return*. $K(x)$ is roughly the probability that spending $\hat{m}(x)$ seconds ends the round; dividing one by the other gives expected rejections per CPU second, and greedily taking the best return first is the natural heuristic for the cost in Equation (3.1). Informally: run the test that is most likely to say "no" per second spent, not the test that is most likely to say "no" overall. This is full cost-cognizance in the sense of Elbaum et al. [19], with the difference that $\hat{m}$ here is measured online from runs the loop was making anyway rather than supplied in advance.

A natural generalisation is to raise the denominator to a power c, which at $c = 0$ recovers ranking on the rejection rate alone and at $c = 1$ recovers Equation (4.2); values in between discount a cost estimate that is noisy early in a session. The implementation and every number in this report use $c = 1$. Tuning c on the pilot and reporting the result on the same data would be a selected maximum, so the sweep belongs to the corpus run (Section 9.2), where the value can be declared in advance.

Exploration. With probability $\varepsilon = 0.15$, a position in the order is filled uniformly at random from the remaining tests instead of greedily. It is in the report rather than left as an implementation detail because it is load-bearing. Without it, a task whose first test rejects every candidate reaches the end of the session having measured exactly one test: the cost estimate never fills in, so the scheduler has nothing to rank by and the hint selector has one fact to choose from. Because ε changes measured numbers it is part of the configuration key. That has a cost worth recording: the random draws are seeded from that key, so two configurations differing in one flag also explore differently,

which makes a single-flag ablation noisier than it needs to be.

4.3.2 Correcting a Marginal Rate

Equation (4.2) ranks each test on its own record. That is a *marginal* rate, and under stop-at-first-rejection the quantity that decides cost is a *conditional* one.

Consider two tests that reject exactly the same candidates. Each has the same record, so each gets the same K, and both sit near the top of the order. Suppose the first of them runs and the candidate *passes* it. The second is now nearly worthless: everything it is known to reject has already been shown not to apply here. The marginal K cannot see this, because it never looks at *which* candidates a test rejected, only how many.

This is the distinction that separates the *total* and *additional* families in test prioritisation [20], with a different notion of coverage: not lines of code, but the set of candidates a test is known to reject. Write $R(x)$ for the set of candidates test x has been observed to reject so far in the session, and let P be the tests that have already run *and passed* on the current candidate. Define

$$\text{overlap}(x \mid a) \;=\; \frac{|R(x) \cap R(a)|}{|R(x)|}, \tag{4.3}$$

and defer x — move it to the back of this round's order — when $\text{overlap}(x \mid a) \geq \tau$ for some $a \in P$, with $\tau = 0.75$ by default and at most one deferral per test per round.

Two properties matter. It **defers and never drops**: a deferred test still runs, it runs last, so an accepting round still executes every test and Definition 3.2 is untouched. And **the order costs nothing to abandon**: it is built one position at a time from the store as it stood when the round began, and construction stops at the first rejection, so the unprobed tail is never materialised. Scheduling cannot change *which* candidates are accepted, so the scheduling claim is a pure cost claim and a difference in repair rate between two schedules is a soundness bug rather than a result.

Equation (4.3) is also what the hint selector uses to drop near-duplicate hints (Section 4.4).

4.4 Selected Hints

4.4.1 The Prompt Is Assembled From Slots

The prompt is never built by string concatenation at a call site. It is assembled from eight numbered slots, so that a configuration changes which slots are filled rather than how a sentence is written, and two configurations can be compared byte for byte.

Table 4.1: The prompt slots. S1–S4 and S7 are identical in every configuration, which is what makes round 1 byte-identical across the grid; the hint mechanism is S5 and S6.

Slot	Contents	When
S1	Role and input/output contract	always
S2–S3	Program label and current source	always
S4	Worked examples, from the task's own public samples	when it has them
S5	**The contract: k selected requirements**	when the store holds ≥ 1 proven test
S6	**Progress line: a count and a pass/fail pattern**	with S5
S7	Output format	always, and always last
S8	Sampling instruction	one sensitivity arm only

4.4.2 What a Hint Is

S5 opens with one header line and then lists the selected tests. Figure 4.2 is the block as the model receives it under the default disclosure.

```
Your patch must produce exactly this output on each of these inputs:

  Input #1 - an estimated 43% of attempted fixes get this input wrong:
    stdin: (verbatim, 2 lines)
<<<
5 3
1 4 2 8 5
>>>
    required: (verbatim, 1 line)
<<<
14
>>>
    your last attempt printed `11`

On the input above, your last attempt scored: fail.
```

Figure 4.2: Slots S5 and S6 for one selected test, at the default disclosure. Everything here is a statement about what the patch must do or about what was observed; nothing names a behaviour to avoid, and nothing names the test itself.

Four properties of this form matter, and each is a rule in the implementation.

No instruction names a behaviour to avoid. The hint says what the patch must do, never what it must not. This is sharper than "prefer positive phrasing": the failure mode is specifically an *instruction together with a named target*. On the frozen log, a failure class the model would otherwise emit 5.4 % of the time was emitted 14.7 % of the time once the prompt named it — a 2.7× priming inversion — while the instruction cut class repetition only from 82.1 % to 70.9 % and grew the prompt from 462 to 1,263 tokens. A *description* is not an instruction, which is why `your last attempt printed `11`` is allowed: it states an observation and names no class. Format constraints in S7

("nothing else", "no prose") stay, because they are identical in every configuration and changing them would break round-1 identity.

Only proven tests are shown. A test whose estimate comes entirely from the prior is a guess, and by the mechanism above a wrong guess does not merely waste tokens, it primes. A test must have rejected at least one candidate this session before it can appear.

The number shown is the posterior rate, and it is worded as an estimate. The figure is $K(x)$ of Equation (4.1) — the same Beta-posterior rate the scheduler ranks by, rounded to the nearest percent. It is *not* the ranking score and *not* the raw frequency $k(x)/n(x)$. Under the default prior $K(x) = (k(x) + 0.5)/(n(x) + 2.5)$, so 43 % is displayed both by a test with $k{=}4, n{=}8$ and by one with $k{=}1, n{=}1$, whose tallies are 50 % and 100 %. The sentence therefore says *an estimated* 43 % *of attempted fixes get this input wrong* rather than counting past attempts, because a tally would be false in both cases. Whether the number is shown at all is a separate configuration (`show_kill_rate`): it is the only thing in the prompt that says which requirement is load-bearing, and it has no analogue in the fact-selection literature, so it is priced rather than assumed.

A test's identifier is never shown. A file name carries nothing the model can act on, and on this benchmark it carries something it should *not* have: contest naming marks the public sample cases, which are the small, highly discriminating ones. Because no prompt names a test, the rule that a prompt never names a test whose content it omits holds by construction.

4.4.3 How Hints Are Selected

At every round the selector ranks proven tests by

$$\text{hintscore}(x) \;=\; K(x) \times \text{legibility}(x) \tag{4.4}$$

and walks the ranking greedily, taking at most k — with $k = 3$ by default.

Cost is deliberately absent. The scheduler and the selector spend different currencies: a scan pays CPU seconds, a prompt pays tokens. A test taking five seconds is expensive for the scheduler and free for the prompt, so carrying $\hat{m}$ into Equation (4.4) would discard the most informative facts for nothing.

Legibility replaces it. legibility(x) is 1 when the test's input is at most 400 characters and 0.45 otherwise. The threshold exists because an input of several megabytes cannot be shown, and showing a truncated prefix of it while asserting a required output produces a *false fact* — the model is told to produce a specific output on an input it

has not seen. The incident that produced the rule: the highest kill-rate test for one task is a 2.3 MB stress input, which is an excellent *schedule* entry and worthless as a *fact*. An illegible input is therefore rendered as a *shape* — its line count, its character count, and its first line verbatim, because on this benchmark the first line almost always carries N and so says whether the failure is a complexity problem or an edge case. An expected output too large to show is *withheld* rather than shaped, since a required value the model cannot read is noise with an assertion attached. The penalty is a factor rather than a veto, so a dominant illegible test can still outrank a legible one when nothing else is close.

Near-duplicate hints are dropped. Two hints that reject the same candidates are one fact spending two slots. The selector skips a candidate hint whose rejection set is already covered by one it has chosen, using the containment of Equation (4.3) and the same threshold τ — but *symmetrically*, unlike in the scheduler:

$$\max\Big(\text{overlap}(x \mid m),\ \text{overlap}(m \mid x)\Big) \ \geq\ \tau \quad \text{for some already-chosen } m \implies \text{skip } x. \tag{4.5}$$

In the scheduler one test has already run and the question is whether the next still adds anything; here neither has run and the question is whether the *pair* is worth two slots, so a narrow hint inside a broad one is dropped whichever way round the two are ranked.

k is a ceiling, not a count. Between the deduplication above and a thin pool of proven rejecters — 55 % of pilot sessions ended with fewer than two — the selector returns fewer than k far more often than it returns k. Every record therefore carries the number of hints that actually reached the prompt alongside the number requested, and the sweep of Chapter 5 is read against the former. A sweep read against the requested k would report a comparison it did not run.

4.4.4 The Disclosure Dial

How much of a test case the prompt reveals is a parameter, not a fixed choice, because it trades two risks against each other. Showing less gives the model too little to work with. Showing more invites it to satisfy *the shown tests* specifically — writing a special case for the displayed input rather than repairing the fault — which produces a patch that passes the hinted tests and fails everything else.

Caster therefore exposes three levels, measured as a dose-response curve in Chapter 6:

The number of hints k and the disclosure level are the same dial seen from two sides, and Chapter 6 reports them as one curve rather than two findings.

Table 4.2: Disclosure levels, from least to most revealed.

Level	What the prompt shows
`sample-only`	**Nothing from the store.** The contract block is withheld in full, so the only worked examples in the prompt are the task's own public samples. This is the floor of the ladder, not a partial disclosure.
`no-required-line`	The selected *inputs*, and a line saying the patch is checked on them — but their expected outputs are withheld.
`full` (default)	The selected inputs, their expected outputs, and an explicit statement that the patch must produce them.

4.4.5 Why This Is Not a Transcript

Three differences follow from the construction. *The prompt does not grow*: at most k hints appear in any round and the bound does not depend on how long the session has run, where a transcript grows linearly in rounds and therefore walks past the point where more context starts to hurt [18]. *The content changes*: which tests have the highest rejection rate moves as the session proceeds, so round 10 can carry entirely different hints from round 3, where a transcript is append-only. *The facts are earned, not asserted*: each hint is backed by a measured record produced by the scheduler that had to run the test anyway, and the hint mechanism adds no executions of its own. Even the one line that reports the previous attempt carries the *last* outcome only — an accumulating record grows the prompt roughly 3× for a benefit measured at zero.

Chapter 5

Experiments

This chapter states what is being asked, on what data, against what, and how an answer is judged. It is written before the results so that every number in Chapter 6 has a meaning fixed in advance rather than one chosen once the numbers arrived.

5.1 Research Questions

Four questions. Each is paired with the outcome that would *refute* it, because a question with no refuting outcome is a label for a group of experiments rather than a question.

RQ1 — Does cost-aware scheduling reduce the cost of rejecting a patch? Does ranking tests by $K/\hat{m}$ — an online rejection rate divided by an online cost — reduce the CPU spent on validation, against the order the benchmark ships, an uninformed random order, the published repair-loop prioritiser FRTP [2], and the classical additional-greedy coverage prioritiser [20]?

Refuted when it is not cheaper than the shipped order beyond the bootstrap interval; or when it is cheaper than the shipped order but not cheaper than FRTP and `coverage`, in which case the contribution is early stopping, which is not ours.

RQ2 — Which part of the rule carries the effect? Three things separate the proposed scheduler from the shipped order: having a memory at all, turning a count of rejections into a rate, and dividing that rate by cost. Which one produces the gain?

Refuted when no single step clears its interval, so the gain is "having any memory" and the score function is ornament.

RQ3 — Do selected hints reduce redundant generation? Does handing the model a small set of concrete requirements, freshly chosen each round, move its proposals off the ideas it has already tried — fewer duplicate patches and at least as many repairs at the same budget — where independent repeated sampling, a prohibition, and a full conversation transcript do not?

Refuted when the duplicate rate does not fall against independent repeated sampling; or when it falls and the repair rate does not rise, in which case the distribution moved somewhere no better.

RQ4 — How much should a hint disclose? Showing a counterexample tells the model what to satisfy. At what level of disclosure does it start satisfying *the tests it was shown* rather than repairing the fault?

Refuted when the measure is flat across every setting, in which case there is no dose and the disclosure axis does not exist.

RQ1 and RQ2 are answered by configurations that change only the test order, and are reported in Chapter 6. RQ3 and RQ4 need configurations that change the prompt, which have not been run.

5.2 Benchmark and Corpus

The corpus is the Python subset of ConDefects [1], chosen for four reasons given in Section 3.4: every test input ships with its expected output, one test is one whole-program execution, the inputs inside a task differ widely in how much work they provoke, and the faults post-date the training cut-off of commonly benchmarked models.

A task is admitted when all of the following hold. The screen runs once, before any configuration, and the admitted set is frozen under a checksum so no later analysis can change it.

Table 5.1: Corpus admission criteria. Applied once, before any configuration runs.

Criterion	Why
Single-file Python fault	The scope of this study; matches a sandbox of one program and one input.
Faulty and reference versions both present	The reference supplies the expected outputs the oracle compares against.
At least one test rejects the faulty version	Otherwise the task is already repaired at round 0 and measures nothing.
The reference passes every test in the pool	A test the correct program fails is an instrument fault; such tests leave the pool.
The reference finishes within the sandbox clock on every pooled test	A test the correct program cannot finish is not a test.
Pool size at or above the declared minimum	A scheduling experiment on a pool of three tests measures nothing about scheduling.

This is a pilot, and the word is meant literally. One hundred tasks were drawn, against a full corpus of roughly 1,244 faults. The pilot exists to establish whether the effects are large enough to be worth a corpus run, and to fix the parameters the corpus run must declare in advance. Three consequences follow and are carried into Chapter 8: intervals are wide, any parameter chosen by a sweep on these 100 tasks is optimistically biased, and the drawn subset is not a random sample of the corpus.

5.3 Configurations

5.3.1 Two Blocks

Configurations fall into two groups, and the difference governs what they cost to run.

Block A changes only the *order* in which tests run. The prompt is identical across these configurations, so the candidate patches can be drawn once and replayed: adding a Block A configuration costs CPU but no model calls. Everything in Chapter 6 is Block A.

Block B changes the *prompt*. The round-t prompt depends on what validation found in rounds 1 to $t-1$, so the draw and the schedule cannot be separated and each configuration needs its own grid of model calls.

One identity ties them together. In round 1 the store is empty, so a prompt with hints carries none and is byte-identical to one without. Round 1 must therefore be the same draw in every configuration, which requires the draw seed to depend on the task, the seed and the round number only — never on the configuration. Deriving it from the full configuration key would make every configuration buy its own round 1 and would destroy the pairing that every comparison below depends on.

5.3.2 Block A — Test Orders

Table 5.2: Block A. Five configurations answer RQ1; three more exist only to split the proposed rule into its parts for RQ2.

Name	Order of tests	Answers
`natural`	The order the benchmark ships — **the incumbent**	RQ1
`random`	Seeded shuffle, reshuffled each round — **the control**	RQ1
`frtp`	Tests that have rejected before, by *count* descending, then the rest [2]	RQ1, RQ2
`coverage`	Additional-greedy over the reference's line coverage [20]	RQ1
`sched`	$K/\hat{m}$, with the marginal-rate correction of Section 4.3.2 — **ours**	RQ1
`cost-only`	$\hat{m}$ ascending; no rejection information	RQ2
`kill-only`	K descending; no cost information	RQ2

Two framing points govern how this table is read.

`natural` is the incumbent, not a neutral control. Sorting by file name is not a scheduling decision anyone made; it is what an implementation gets for free, and therefore what most implementations do. It is also not arbitrary: contest test directories put the small, public sample cases first, and those are both cheap and highly discriminating, so the shipped order is *better* than chance. The uninformed control is `random`, and reporting both separates "better than nothing" from "better than what people actually run".

`frtp` is the published baseline, implemented as published. It ranks on the rejection *count*, which is what FRTP ranks on — not on a rate, and not on anything divided by cost. A baseline weakened to flatter the proposal answers nothing, and as Chapter 6 shows, this one is hard to beat.

What is not in the table. Earlier drafts of this study carried five more Block A configurations — alternative cold starts, a bucketed score, a zero-exploration variant. They test whether the answer survives routine changes to our own parameters, which is a question about this implementation rather than about the method, and reporting them beside the five above gave thirteen rows of which eight answered nothing a reader asked. They remain in the tool and in the run output; they are not reported here.

5.3.3 Block B — Prompts

Table 5.3: Block B. Not yet run. All use the proposed scheduler unless the row says otherwise.

Name	What the prompt carries after round 1	Answers
`no-mem`	Nothing; every round redraws from scratch — **the baseline**	RQ3
`chat`	The full conversation transcript, CHATREPAIR-style [6]	RQ3
`exclusion`	The same facts phrased as a prohibition	RQ3
`hints`	k=3 selected requirements, shipped test order	RQ3
`sched+hints`	k=3 selected requirements **and** the proposed scheduler	RQ3
`hints-k1`, `hints-k2`	$k \in \{1, 2\}$ instead of 3	RQ4
`hints-sample`, `hints-noreq`	Lower disclosure levels (Table 4.2)	RQ4

`no-mem` is the baseline for RQ3, and it is a strong one. It redraws independently every round, which is exactly *independent repeated prompting*. Chen et al. [7] measured that strategy at 23 bugs repaired against CHATREPAIR's 18 at an equal budget of 24 model calls, with 65 % of the conversational system's patches being duplicates. The obvious iterative strategy is already published as *worse* than simply drawing again, so Block B must beat `no-mem`; beating `natural` would prove nothing on the generation side.

Table 5.4: The 2 × 2 behind RQ3: each mechanism alone, and both together, on the same tasks and seeds.

	Shipped test order	**Cost-aware scheduling**
No hints	`no-mem` (baseline)	`sched`
Selected hints	`hints`	**`sched+hints`**

5.4 Metrics

The study does not collapse into one figure of merit, because the two halves of the loop spend different resources and a composite would let one pay for the other.

Validation side (RQ1, RQ2). CPU seconds per session is the primary metric — the `getrusage` delta summed over the tests actually run, paired per task and seed. Success is `sched` below all four baselines beyond the bootstrap interval. **Test runs per session** is reported alongside, because the prioritisation literature reports executions, but is not a success criterion on its own. **CPU per test run** is the ratio of the two: it separates "found the rejection in fewer steps" from "found it in cheaper steps", and it is where the cost term lives.

Generation side (RQ3, RQ4). The primary metric for RQ3 is the **duplicate rate**, $1 -$ distinct/total patches over normalised patch text; it must fall against `no-mem`. **Sessions repaired** within budget must rise, or at minimum not fall, wherever it does. **Prompt and completion tokens** are reported for every Block B configuration, since the LLM-repair literature prices repair in tokens. The primary metric for RQ4 is **leakage**: of the patches rejected at all, the share whose rejecting tests are *disjoint* from what the prompt showed — the patch satisfied everything it was told and died elsewhere. It should fall as disclosure rises, or the disclosure axis does not exist.

One quantity is guaranteed rather than measured. By Definition 3.2 a candidate is accepted only after surviving every usable test, so the share of sessions repaired and the round of the first repair are *identical* across every Block A configuration — on every task and seed, not merely on average. The experiment asserts that equality and halts if it is violated. A difference there would be a soundness bug, not a finding.

5.5 How the Numbers Are Produced

5.5.1 The Tool

Caster is a Python package of command-line verbs. The ones needed to reproduce the pilot are `freeze` (apply the screen, write the frozen corpus), `propose` (fill the candidate store), `audit` (one full-scan pass recording, for every candidate, which tests reject it and what each costs), `evaluate` (walk the task $\times$ seed $\times$ configuration grid, resumably), then `merge`, `analyse` and `report`. The package ships 263 unit tests that run without a network, a model or the benchmark.

5.5.2 How Cost Is Measured

Cost is the delta of `getrusage(RUSAGE_CHILDREN)` across one test run: CPU seconds, never wall clock. Wall clock on a machine running several workers is contaminated by its siblings, which would make the learned cost estimate a function of machine load rather than of the test. Because `getrusage` accounting is per process, parallelism is process-per-worker and the sandbox refuses to run under a thread pool rather than let one worker's CPU be attributed to another.

Three rules keep the measurement honest.

1. **The cost table comes from the audit pass, not from any configuration.** If a configuration could populate it, the table would become a function of which configuration ran first and every later comparison would inherit the bias.

2. **No (patch, test) pair is executed twice.** Outcomes are memoised, and the identity (fresh runs − timeout re-runs) + cache hits = $\sum$ test calls is instrumented per scan rather than asserted in prose. A stored outcome that disagrees with a fresh one halts the run.

3. **Timeouts are quarantined.** A timed-out test is never served from the cache and never enters the cost model: a correct-but-slow program times out too, and entering timeouts at the clock value would let a handful of them dominate the cost estimate.

5.5.3 Statistics

Every comparison is **paired** at the level of a (task, seed) cell under common random numbers, so two configurations see the same round-1 candidate on the same task; an unpaired comparison on 100 tasks with this corpus's cost spread would be dominated by which tasks landed where. **Intervals** come from a cluster bootstrap over tasks — resampling whole tasks with replacement, 10,000 times — because the three seeds of one task share the task, its pool and its cost structure, and bootstrapping over cells would report an interval too narrow by roughly $\sqrt{3}$. **Tests** are Wilcoxon signed-rank over per-task totals, since the distribution of per-task CPU is heavily right-skewed and a t-test would be driven by a few expensive tasks. **Effect sizes** are reported as a percentage change in total CPU *together with* the number of tasks going each way, because an aggregate win driven by three expensive tasks is a different claim from a win on eighty — and Chapter 6 reports both for the contrast where they disagree. Four contrasts are declared in advance and carry a **Holm correction**: `sched` against each of `natural`, `random`, `coverage` and `frtp`. The ablation ladder is reported with intervals and labelled exploratory.

Chapter 6

Results

This chapter reports the Block A pilot: the configurations that change only the *order* in which tests are run. Block B — the configurations that change the *prompt* — has not been run, and its three sections at the end of the chapter state what will go in them.

Everything below comes from one run of 97 tasks × 3 seeds × 7 configurations, a budget of 20 rounds per session. Section numbers in this chapter match the research questions of Section 5.1.

6.1 What the Pilot Measured

6.1.1 The Tasks

One hundred tasks passed the admission screen of Table 5.1. Ninety-nine produced sessions. Two of those are missing at least one (task, seed, configuration) cell, because the audit pass had not recorded an outcome for every (patch, test) pair they needed, and the merge refuses to guess. The remaining 97 **tasks are complete in every configuration and every seed**, and every comparison below is made on exactly those 97, paired task by task.

Table 6.1: The pilot universe.

Tasks admitted / ran / complete in every configuration	100 / 99 / 97
Seeds per task	3
Round budget per session	20
Sessions measured (task × seed × configuration)	3,783
Test cases in the admitted pools	2,521
Pool size per task — median (min–max)	25 (8–58)
Distinct candidate patches in the audit matrix	1,167

6.1.2 Is There Anything for a Cost Term to Do?

The scheduling rule divides by an estimate of what a test costs. That division can only matter if tests within one task differ in cost. If every test in a task took the same time, ranking by "rejections per second" and ranking by "rejections" would produce

the same order, and the whole cost half of this work would be an elaborate way of writing FRTP.

So the first thing to check is the spread, and the answer is: *large across the corpus, modest inside the median task.*

Table 6.2: The two distributions the method depends on. "CV" is the coefficient of variation — the standard deviation divided by the mean — which is 0 when every test in a task costs the same and grows as they diverge.

Cost of running one test case, over all 2,378 *tests with a measured cost*	
median	0.036 s
90th percentile	1.41 s
99th percentile	6.32 s
maximum	10.14 s
median CV *within* a task	**0.159**
share of tasks with within-task CV ≥ 0.3	35 %
How often a test rejects a candidate, from the full-scan audit	
mean rejection rate	0.376
median rejection rate	0.250
tests that never rejected anything	31.2 %
Median within-task correlation between log(**cost**) **and rejection rate**	**+0.11**

Three readings matter.

Across the corpus the spread is enormous; inside a task it usually is not. The 99th-percentile test costs 177 times the median one. But a schedule only ever chooses among the tests of *one* task, so the relevant number is the within-task CV, whose median is 0.159 — small. Figure 6.1 shows the whole distribution: only about a third of tasks have a spread the cost term can act on, which predicts most of what follows — it should help a lot on a minority of tasks and do nothing on the rest.

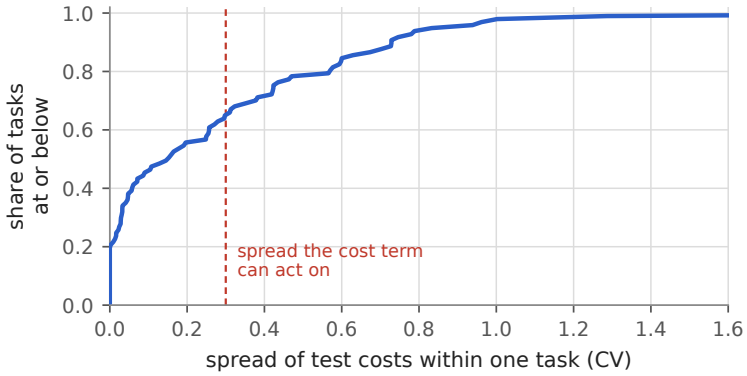


Figure 6.1: How much test costs vary *within* a single task, as a cumulative share of the 97 tasks. Two thirds sit below a CV of 0.3, where there is little for a cost term to exploit.

A third of the tests never reject anything. They are pure cost. Any memory of past rejections finds them and pushes them down, which is why *having a memory at all* turns out to be the dominant effect in Section 6.4.

Expensive tests are, if anything, slightly *better* at rejecting. The median within-task correlation between log cost and rejection rate is +0.11, so ranking on rejection alone is mildly drawn towards the expensive end of the pool. Dividing by cost is a correction to that pull, not a refinement of an unbiased ranking.

6.2 RQ1a — Is Scheduling the Oracle Worth Doing?

The first question is the coarse one: does deciding the test order at all beat not deciding it? Three configurations answer it. `natural` is the order the benchmark ships, which is what an implementation gets for free. `random` is a reshuffle each round, which is what "no information" looks like. `sched` is the proposed rule. The published prioritisers are deliberately held back to Section 6.3, because on this scale they would be invisible.

Table 6.3: Scheduling against not scheduling, over the 97 complete tasks (291 sessions per configuration, 20 rounds each). Intervals are 95 % cluster bootstrap over tasks, 10,000 resamples; p is a Wilcoxon signed-rank test over per-task totals with a Holm correction.

Configuration	Total CPU (s)	CPU / session	Test runs / round	CPU / test run
`natural` (incumbent)	7,383	25.37	7.16	0.298
`random` (control)	8,331	28.63	6.08	0.397
`sched` (ours)	**2,893**	**9.94**	2.43	0.344

Paired contrast	Δ CPU	95 % CI	Δ test runs	p (Holm)
`sched` vs `natural`	−60.8 %	[−73.3, −36.3]	−66.0 %	1.7×10^{-8}
`sched` vs `random`	−65.3 %	[−77.4, −45.3]	−59.9 %	1.0×10^{-9}
`random` vs `natural`	+12.8 %	[−36.8, +97.6]	−15.2 %	0.77

The answer is yes, by a factor of two and a half. The same 291 sessions cost 7,383 seconds under the shipped order and 2,893 under `sched` — a reduction of 60.8 %, with an interval nowhere near zero and 52 of 97 tasks going the same way (only 13 go the other). Against the uninformed control the reduction is 65.3 %.

The shipped order is not a straw man, and `random` shows it. Shuffling the pool is *worse* than the shipped order on CPU (+12.8 %), although the interval is wide and the difference is not significant. Contest test directories put the small, hand-written sample inputs first, so sorting by file name already puts cheap, discriminating tests near the front. Beating `natural` by 60 % is therefore a real result rather than an artefact of

a weak comparison — and the fact that `random` runs *fewer* tests than `natural` while costing *more* is the first sign in this chapter that the two units of account can disagree.

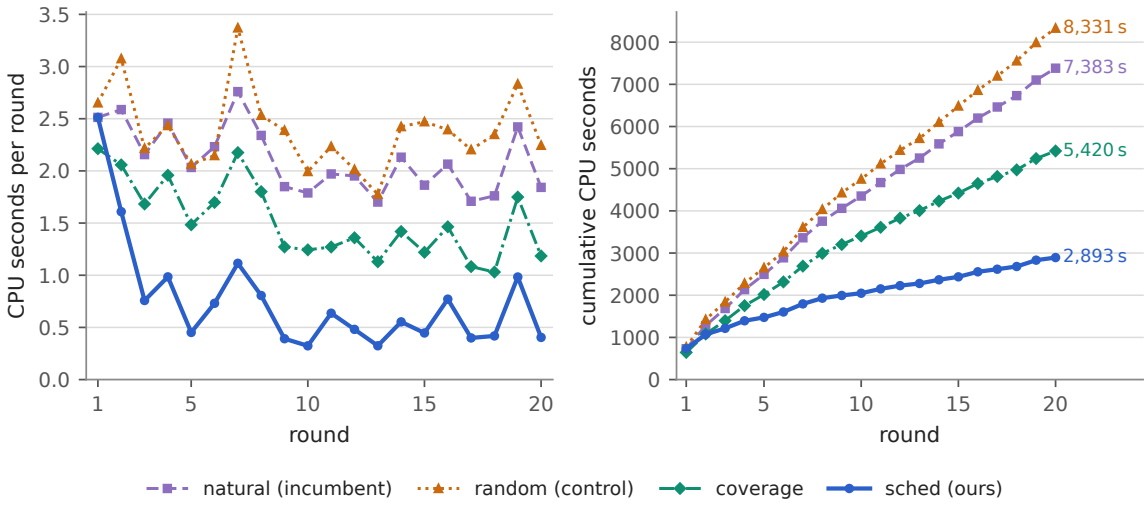


Figure 6.2: Left: mean cost per round. Round 1 is identical for `natural` and `sched` by construction — the store is empty, so both run the shipped order. The separation appears in round 2 and is complete by round 4; the two unscheduled lines never improve, because neither has a memory, so round 20 costs what round 2 cost. `coverage` is drawn here as well, because it separates cleanly on this scale: it is a scheduler, so it sits below the unscheduled pair, but it reads only static information and so never learns. Right: the same data summed, which is the quantity a deployment pays.

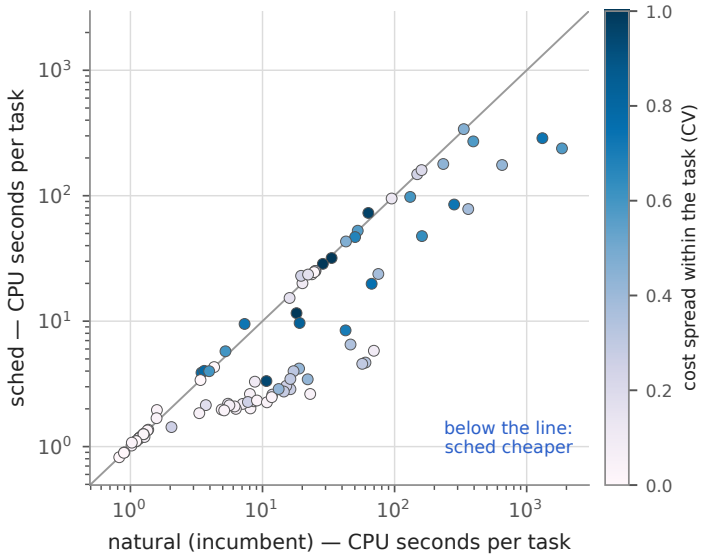


Figure 6.3: Each dot is one task; both axes are total CPU over three seeds, on logarithmic scales. Almost every task sits below the diagonal, many by an order of magnitude, and the gain is not confined to a few expensive tasks.

The gain arrives in one round and never reverses. Figure 6.2 shows where it comes from. Round 1 costs 2.51 s in both `natural` and `sched`, because the store is

empty and there is nothing to rank by. By round 3 `sched` is at 0.76 s and `natural` is still at 2.16 s. Splitting the budget into blocks of five rounds gives −40.9 %, −69.2 %, −74.6 % and −69.6 %: the first block is diluted by the round in which no configuration knows anything, and the rest are flat, because the rejection history saturates quickly.

Table 6.4: Soundness. By Definition 3.2, every Block A configuration must repair exactly the same sessions on exactly the same rounds. This is asserted by the experiment rather than reported as a finding: a difference here would be a bug.

Configuration	Sessions repaired	First-accept round (mean)
`natural`, `random`, `coverage`, `frtp`, `sched`, and both ablation configurations	47.77 %	2.99

Soundness. All seven configurations agree on every cell. Everything in this chapter is therefore a pure cost claim: nothing trades repairs for seconds.

6.3 RQ1b — Against the Published Prioritisers

Section 6.2 established that scheduling pays. It did not establish that *this* scheduling rule is the reason, because any rule with a memory would clear that bar. The real comparison is against the two prioritisers the literature already offers: `coverage`, the additional-greedy rule over reference line coverage, and `frtp`, which ranks tests by how many patches each has already rejected [2]. Both are in Table 6.5 on their own scale.

Table 6.5: The three scheduling rules, on the scale where they actually differ. The **CPU per test run** column is the one to read: it separates "found the rejection in fewer steps" from "found it in cheaper steps". The last row is not a contrast about `sched`; it is there because the size of `frtp`'s own gain over the incumbent is what makes the first two rows worth reading.

Configuration	Total CPU (s)	CPU / session	Test runs / round	**CPU / test run**
`coverage` ($\mathrm{RTP}_{\mathrm{add}}$)	5,420	18.63	5.78	0.271
`frtp` (rejection count)	2,979	10.24	2.15	0.401
`sched` ($K/\hat{m}$)	**2,893**	**9.94**	2.43	**0.344**

Paired contrast	Δ CPU	95 % CI	Δ test runs	p (Holm)
`sched` vs `coverage`	−46.6 %	[−62.3, −20.4]	−57.9 %	1.1×10^{-4}
`sched` vs `frtp`	−2.9 %	[−11.0, +4.9]	+13.1 %	0.026
`frtp` vs `natural`	−59.7 %	[−73.3, −33.0]	−70.0 %	8.4×10^{-10}

`coverage` is not competitive, and the reason is structural. It uses information — which lines of the program each test exercises — but *static* information, fixed before the session begins. It cannot learn from what this session has just seen, so its line in

Figure 6.2 is flat, it ends 46.6 % above `sched`, and it does not clear the incumbent by a separable margin either (-26.6 %, interval $[-56.7, +7.4]$). In a setting where every test is one whole-program execution, knowing which lines a test touches says much less about whether it will reject a patch than simply remembering whether it has.

`frtp` is the real opponent, and on the whole-session total the two are not separated. The point estimate is -2.9 % with an interval from -11.0 % to $+4.9$ %. Taking that number alone and stopping would miss the two things the rest of this section is about: *when* the difference appears, and *where*.

6.3.1 When: the Cost Term Needs the Cost Table to Fill In

The two halves of $K/\hat{m}$ do not become usable at the same time. A rejection count is informative after one observation — a test either rejected the patch or it did not. A cost estimate is a *mean*, and a mean over one or two samples of a quantity that varies by orders of magnitude is mostly noise. So for the first few rounds, `sched` is dividing by a number it does not yet know, while `frtp` is already ranking on something it does.

Table 6.6 splits the twenty-round budget into four blocks of five and runs the paired comparison inside each.

Table 6.6: `sched` against `frtp`, by block of rounds. Paired over the same 97 tasks, same intervals as everywhere else. The whole-session figure of -2.9 % is the average of these four, weighted by the CPU each block carries — and the first block carries half of it.

Rounds	`frtp` (s)	`sched` (s)	Δ CPU	95 % CI	`sched` cheaper on
1–5	1,444.8	1,474.5	+2.1 %	$[-1.5, +7.5]$	23 / 62 differing
6–10	604.4	573.1	−5.2 %	$[-16.6, +5.0]$	14 / 55
11–15	434.1	388.3	**−10.6** %	$[-23.0, +6.2]$	12 / 49
16–20	495.4	456.8	−7.8 %	$[-19.2, +3.8]$	14 / 49

The table and the figure say the same thing: `sched` *pays* about 2 % early for an estimate it does not yet have, breaks even at round 10, and is then 5 to 11 % cheaper for the rest of the budget. As a running total the deficit peaks at +34 s in round 6 and the surplus reaches −86 s by round 20.

Two cautions keep this honest. None of the four intervals in Table 6.6 excludes zero, so the pattern is a trend rather than four separate results. And the later blocks are computed on the sessions that have *not* yet been repaired, which are the harder ones — although that selection applies identically to both configurations, since by Definition 3.2 they repair the same sessions on the same rounds.

The practical reading is that a twenty-round budget is roughly the point at which the cost term starts to pay, and this pilot stops there. That is a direct argument for a longer budget in the corpus run, not a hedge.

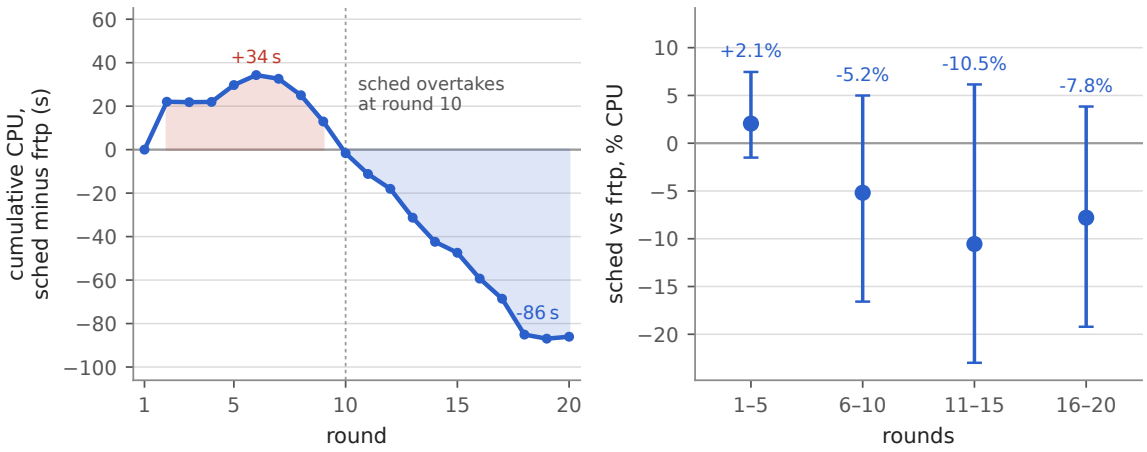


Figure 6.4: Both panels plot a *difference*, deliberately. Drawn as two absolute curves these two configurations sit on top of each other near 0.4 s per round, on an axis that has to reach 2.5, and a ten-percent gap becomes a line width. Left: the running difference in seconds. `sched` pays up to 34 s while its cost estimates are still forming, breaks even at round 10, and then separates at a steepening rate, finishing 86 s ahead with the gap still widening when the budget ends. Right: the same comparison in blocks of five rounds, with cluster-bootstrap intervals.

6.3.2 Where: the Win Is Concentrated, Not Spread

Table 6.7: `sched` against `frtp`, task by task. The two schedules produce an identical cost on a third of tasks. Where they differ, `frtp` is more often the cheaper one — but `sched`'s wins are much larger than its losses, so the total goes the other way.

Tasks where the two cost exactly the same	34 / 97
Tasks where `frtp` is cheaper	45
Tasks where `sched` is cheaper	18
Total CPU `sched` saves on the tasks it wins	188 s
Total CPU `sched` loses on the tasks it loses	102 s
Net	**−86 s** (−2.9 %)

That shape — many small losses paid for by a few large wins — is exactly what dividing by cost is supposed to produce. The rule gives up a little on tasks where every test costs the same, in exchange for avoiding the expensive test on tasks where one exists. It behaves like insurance, and a pilot of 97 tasks has too few expensive tasks to price insurance precisely.

Section 6.1 predicted *which* tasks those would be: the ones whose test costs actually differ. Table 6.8 splits the 97 into three equal groups by that spread.

The gradient runs the right way: a consistent small penalty where there is no spread, and the only negative sign in the high-spread group, which is also the group carrying 79 % of the corpus's total validation cost. Figure 6.5 is the same fact as a scatter — the cloud hugs the diagonal, and the points that leave it are the dark ones.

Table 6.8: `sched` against `frtp`, split by how much test costs vary within the task. The ordering of the three rows is the prediction of Section 6.1; the intervals are too wide for any one row to stand alone.

Within-task cost spread	Tasks	CV range	`frtp` (s)	`sched` (s)	Δ CPU [95 % CI]
Low	33	0.00–0.03	117	118	+1.2 % [+0.3, +3.4]
Medium	32	0.04–0.31	567	573	+1.1 % [−0.0, +4.3]
High	32	0.32–2.64	2,295	2,201	**−4.1** % [−13.8, +6.4]

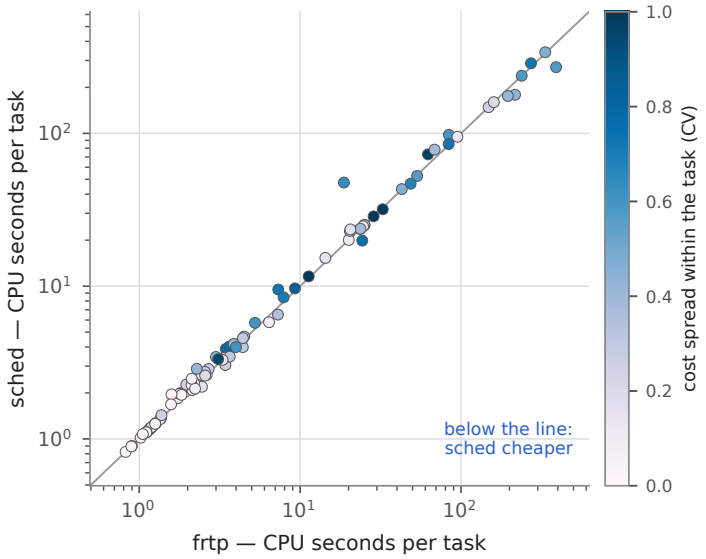


Figure 6.5: `sched` against `frtp`, one dot per task, both axes total CPU over three seeds on logarithmic scales. Compare with Figure 6.3, which is the same plot against the incumbent on the same scales: there, the cloud has left the diagonal entirely.

What the two units say. `sched` runs 13.1 % *more* test executions than `frtp` and still spends less CPU, because each execution is cheaper: 0.344 s against 0.401 s, a drop of 14 %. That column is where the cost term lives, and it is the clearest single piece of evidence that the mechanism is the one claimed rather than a lucky ordering — it is the only configuration in Table 6.5 that trades runs for seconds in that direction.

It also means that a study reporting executions alone would have concluded that `sched` is 13 % *worse* than `frtp`. The prioritisation literature mostly reports executions (Section 7.2).

6.4 RQ2 — Which Term Carries the Effect

`sched` adds three things to the shipped order: a *memory* of what each test has done, a *rate* computed from that memory instead of a raw count, and a *division by cost*. Going straight from `frtp` to `sched` changes two of them at once, so the comparison cannot attribute the effect. Table 6.9 changes one thing per step.

Table 6.9: The ablation ladder. Each row changes exactly one thing relative to the row's starting configuration. Paired over the same 97 tasks.

Step	What changes	Δ CPU	95 % CI	Δ runs
`natural` → `cost-only`	cost enters, no rejection history	−47.8 %	[−65.1, −19.1]	−47.7 %
`natural` → `kill-only`	rejection rate enters, no cost	−59.1 %	[−73.0, −32.5]	−70.5 %
`natural` → `frtp`	the published baseline's step	−59.7 %	[−73.1, −33.2]	−70.0 %
`frtp` → `kill-only`	a **count** becomes a **rate**	+1.4 %	[−0.8, +4.6]	−1.9 %
`cost-only` → `sched`	the cost rule gains a rejection rate	−25.0 %	[−40.7, −6.7]	−35.1 %
`kill-only` → `sched`	the rate is divided by cost	−4.2 %	[−12.0, +3.4]	+15.3 %

Having a memory is almost the whole effect. The three steps out of `natural` are all large and all clear of zero. Either signal on its own — cost, or rejection history — recovers most of the win, and rejection history recovers slightly more than cost.

Turning the count into a rate buys nothing here. `frtp` → `kill-only` is +1.4 % with an interval that includes zero. A rate is the first quantity that reads $n(x)$ — the evidence that a test ran and did *not* reject, which the FRTP family discards — so the step was expected to pay. At a budget of 20 rounds it does not. The likely reason is visible in Table 6.2: with 31 % of tests never rejecting anything and a median of 25 tests per pool, the count and the rate order the pool the same way long before the session ends. A longer budget or a larger pool is where the two would separate, and the pilot has neither.

Dividing by cost is directionally right and not yet separated. −4.2 % on top of the rate, with an interval from −12.0 % to +3.4 %. The execution column is again the tell: the step pays 15.3 % more executions for that reduction, so it is genuinely trading runs for seconds rather than simply finding better tests.

The two signals are not substitutes. Adding a rejection rate to a pure cost rule is worth −25.0 %, clear of zero. Adding cost to a pure rate rule is worth −4.2 %. The asymmetry says the rejection signal is the primary one and the cost term is a correction on top of it — which is how the rule is written, $K/\hat{m}$, and not the other way round.

6.5 RQ3 and RQ4 — Not Yet Run

Not yet measured. Block B — the configurations that change the prompt — has not been run. Each needs its own grid of model calls, because the round-t prompt is a function of what validation found in rounds 1 to $t-1$, so the candidates cannot be drawn once and reused the way Block A reuses them.

Table 6.10 is laid out with its final shape and units. RQ3 asks whether the duplicate rate falls against `no-mem` while the repair rate does not; RQ4 reads the same configurations' *leakage* — the share of rejected patches whose rejecting tests are disjoint from

what the prompt showed — against the number of hints actually rendered, and the 2×2 of Table 5.4 asks whether `sched+hints` beats the better of the two mechanisms alone.

Table 6.10: Block B, not yet run. `no-mem` is the baseline — independent repeated prompting, which the literature already reports as stronger than a conversation transcript.

Configuration	Dup. rate	Repaired	First accept	Leakage	Compl. tokens
`no-mem` (baseline)	—	—	—	—	—
`chat` (transcript)	—	—	—	—	—
`exclusion`	—	—	—	—	—
`hints`	—	—	—	—	—
`sched+hints`	—	—	—	—	—
`hints-k1` / `hints-k2`	—	—	—	—	—
`hints-sample` / `hints-noreq`	—	—	—	—	—

6.6 Summary of Findings

1. **Scheduling the oracle is worth doing.** Against the order the benchmark ships, the proposed scheduler removes 60.8 % of validation CPU ($[-73.3, -36.3]$) and repairs exactly the same bugs on exactly the same rounds.

2. **Almost all of that comes from having a memory, not from the score.** Any configuration that remembers something — a count, a rate, or a cost — lands within a few percent of the others and roughly 60 % below the incumbent. Turning a count into a rate is worth +1.4 % ($[-0.8, +4.6]$): at 25 tests per pool and 20 rounds, the two order the pool the same way.

3. **The cost term pays late, and the budget ends too early to price it.** Over a whole session it is 2.9 % cheaper than FRTP ($[-11.0, +4.9]$). Split by block it is +2.1 % over rounds 1–5, while the cost estimates are still forming, and −10.6 % over rounds 11–15; the running total crosses over at round 10 and is still widening at round 20.

4. **It wins where the theory says it should.** The gain concentrates in the third of tasks whose test costs actually differ (−4.1 % there, +1.2 % where costs are uniform), and it pays 13 % more executions for 14 % cheaper ones. Ranked by executions FRTP wins; ranked by CPU the order reverses, so a result reported in executions alone would have concluded the opposite.

Chapter 7

Discussion

7.1 When the Method Does Not Help

A method is more useful when its failure conditions are written down than when the next person discovers them. The scheduler beats the incumbent almost everywhere, but the part of it that is *new* — the division by cost — needs three conditions, and the pilot gives evidence on all three.

Table 7.1: The three conditions under which the cost term is expected to pay, and what the method degrades to when each fails.

Condition	Measured by	What happens when it fails
C1. The pool is large enough	$\lvert P\rvert$, known before any run	With a handful of tests there is little to order, and a candidate that survives pays for the whole pool whatever the order. Every configuration converges to one schedule.
C2. Test costs are not uniform	The within-task coefficient of variation of per-test CPU (Table 6.2)	$\hat{m}$ becomes a constant, dividing by it changes no ranking, and the rule collapses to FRTP with a denominator.
C3. The task is hard enough for the proposer	How many rounds the loop needs on that task	A task repaired in one or two rounds never builds a history, so there is nothing to schedule by and nothing to select hints from.

C2 is the one the pilot can speak to, and it holds up. The within-task cost spread is the variable that splits Table 6.8, and the comparison against FRTP runs $+1.2\,\%$, $+1.1\,\%$, $-4.1\,\%$ across the three groups. That is the predicted ordering: the cost term costs about one percent where there is no spread and is the only group where it wins where there is. None of the three intervals excludes zero, so this is the *shape* of the prediction confirmed rather than the prediction itself.

The quantity that matters is the spread *within* a task, because a schedule only ever chooses among the tests of one task: a corpus of individually uniform tasks looks heterogeneous in aggregate and offers the scheduler nothing. Here the aggregate spread is enormous — the 99th-percentile test costs 177 times the median — while the median

within-task CV is only 0.159, so two thirds of these tasks are in the regime where C2 fails.

C3 dilutes rather than harms. A corpus of easy tasks does not make the method look bad; it makes every configuration look identical and pulls a corpus-wide average towards no effect — visible here in the 34 of 97 tasks on which `sched` and `frtp` cost exactly the same. Difficulty is therefore a stratification variable, not a caveat.

Note that C1 and C2 are properties of the frozen pool, known before any candidate exists, so splitting on them stratifies a fixed population. C3 is an *outcome*, and admitting tasks on the basis of it would select the corpus on the result — so the pilot reports the whole-corpus number as the headline and the split as a secondary reading, never the other way round.

7.2 Reporting Seconds Rather Than Executions

The prioritisation literature reports the number of test executions rather than elapsed time, for a good reason: the largest study of patch validation to date counts executions precisely because elapsed time is unstable across environments [22]. Time depends on the machine, on what else is running, and on the interpreter; a count does not.

The problem is that the reproducible unit and the deployed unit are different quantities, and the choice between them is not neutral when test costs vary. The pilot produced the textbook case. Against FRTP, the proposed scheduler runs 13 % *more* test executions and spends 3 % *less* CPU. Reported in executions it lost; reported in seconds it won. A deployment pays for seconds.

This report therefore gives **both units for every configuration** — it costs nothing, since a loop that measures time is already counting executions — and reports CPU rather than wall clock, which on a machine running several workers is contaminated by siblings. It also reports the **ratio**, CPU per execution, because that is what separates "found the rejection in fewer steps" from "found it in cheaper steps". In Table 6.5 that column is the whole mechanism: 0.401 s per run for FRTP against 0.344 s for `sched`.

None of this contradicts Lou et al.'s reasoning. Their setting — Java projects with method-level suites — is one where test *selection* is available: a test can be skipped entirely because no changed method reaches it, and there they find selection to dominate prioritisation. Here a test is one whole-program execution on one input, so there is no call graph to cut and nothing to select away. Prioritisation is the only lever.

7.3 Composing with the Other Two Families, and What Each Costs to Adopt

The method changes only two things: which test runs first, and what goes into the prompt. It does not change how a test is executed, and it does not change whether a candidate is executed at all. That makes it composable with the other two families of validation-cost reduction, each of which reduces a different factor of Equation (2.1).

Execution acceleration makes each run cheaper — UniAPR [9] and Express-APR [10] reuse a warm runtime and adapt mutation-testing techniques, reporting up to 137×. A scheduler is indifferent to how a test is executed: it ranks by measured cost, so if acceleration changes the costs it re-learns them. One interaction cuts in a useful direction: acceleration typically removes a fixed per-run overhead, which *increases* the relative spread between cheap and expensive tests, and that spread is exactly what the cost term needs. **Execution-free verification** removes candidates before they are run; R2E-Gym [11] finds learned and test-based verifiers complementary, so a learned verifier can filter cheaply and the scheduler then decides how to spend execution on the survivors. **Selected hints** also reduce the number of candidates, but earlier: they change what is generated rather than filtering it afterwards.

They do not cost the same to adopt. Acceleration needs an integration with one language runtime — class reloading, state resetting — and the correctness argument that comes with reusing a dirty runtime. Execution-free verification needs training data, a training run, a second model to serve, and a standing question about distribution shift. Search over repair attempts [13] spends extra model calls per bug, which increases the resource this work is trying to reduce. The scheduler needs three counters, one sort per round, and a timer around a call the loop already makes (Section 4.1.2).

It is therefore the cheapest of the four to try and the easiest to withdraw: if it does not help it degrades to the ordering the loop already used, and nothing has to be retrained when the proposer is swapped, because the statistics are of the task and the session rather than of the model. That is also an argument about where to start. Acceleration and verifiers have larger ceilings and both require a project; a scheduler is an afternoon, and it establishes whether the pool has any cost structure worth exploiting before anyone commits to the larger work — 60 % of validation CPU here was removed by three counters.

Simplicity is a claim that has to survive the ablation, and it only partly did. Table 6.9 shows a plain rejection count — Frtp, published in 2013 — already captures nearly the whole effect here. The honest statement of the contribution is therefore narrower than the headline: scheduling matters enormously, and the cost term adds a further amount that grows over a session and concentrates where the theory says it should. Establishing that the second part is real needs the full corpus.

7.4 What the Pilot Can and Cannot Settle

A pilot of 97 tasks is a screening instrument: it can show that an effect exists and is large enough to justify a corpus run, and fix the parameters that run must declare in advance. It cannot establish a magnitude.

Settled: scheduling the oracle is worth doing ($-60.8\,\%$, no cost in repairs), and memory rather than the score is the dominant term. **Not settled:** whether dividing by cost beats FRTP at corpus scale, whether a rate beats a count at a larger budget, and anything on the generation side.

One caution applies to the cost exponent c and the hint count k: a value selected on the data it is reported on is optimistically biased, so both are declared before the corpus run (Section 9.2).

Chapter 8

Threats to Validity

8.1 Internal Validity

The policy influences the data it then learns from. Within a session the schedule decides which tests run, which decides what the store contains, which decides the next schedule. Two things limit the damage. Round 1 is identical across configurations under common random numbers, so none starts from an advantage; and the per-test cost table is established by a dedicated full-scan audit before any configuration runs, so no configuration's ordering can contaminate the costs every configuration is scored against.

Instrument faults that produce numbers rather than errors. Three failure modes would yield plausible wrong values instead of a crash: a (patch, test) pair executed and counted twice, a cached outcome disagreeing with a fresh one, and a timeout entered into the cost model at the clock value. Each is instrumented — an accounting identity checked per scan, a halt on disagreement, and quarantine of timeouts — rather than asserted in prose.

Two tasks were dropped after the run. The merge audit refused two of the 99 tasks that produced sessions, because the audit pass had not recorded an outcome for every (patch, test) pair they needed. They are dropped from every configuration rather than some, so the comparison stays paired — but the admitted set and the analysed set are not the same 100 tasks.

8.2 External Validity

One benchmark, one language. All results come from the Python subset of ConDefects: single-file competitive-programming submissions with whole-program, stdout-compared tests. Two consequences follow. Test *selection* is unavailable, so prioritisation is the only scheduling lever, whereas in project-scale Java repair selection is available and has been found to dominate [22]. And contest pools are deliberately heterogeneous in cost, which is favourable to the cost term — an interpreted language exaggerates the spread further. A uniform-cost corpus would show less, which is condition C2 of Section 7.1.

One proposer model, at pilot scale. Every candidate comes from `qwen2.5-coder:7b` at one temperature, so the rejection structure the store learns is one model's: which tests are discriminating depends on which mistakes the model makes. The budget of twenty rounds also truncates the learning curve, and Section 6.3.1 shows that it truncates it at the worst possible place: `sched` only overtakes `frtp` at round 10 and the gap between them is still widening when the budget ends. A longer budget would very likely show a larger effect, but the pilot cannot say how much larger, and the direction of that bias should be read as working *against* the reported $-2.9\,\%$ rather than for it.

The 100 tasks are a draw, not a random sample. They were taken from a corpus of roughly 1,244 faults by the pool order fixed at freeze time. Nothing in the analysis corrects for that, and the full-corpus run is the only thing that removes it.

8.3 Construct Validity

The per-test cost table in the analysis is reconstructed. The report's corpus statistics (Table 6.2) recover each test's frozen cost by least squares from the replayed rounds, because a round's CPU is the sum of the costs of the tests it ran. This is exact where a test appears in enough rounds and noisy where it appears in few. It affects the descriptive statistics and the cost-spread split of Table 6.8; it does not affect any head-to-head total, which is summed from the rounds themselves.

"Duplicate" is defined by normalised text. Two candidates that differ in a variable name but express the same repair hypothesis are not duplicates under this definition, although they are equally wasteful. The duplicate rate is therefore a *lower bound* on redundancy. The same caveat applies to the 65 % figure reported for ChatRepair [7], which this work uses as its reference point.

Cost is CPU; acceptance is plausibility. A deployment paying for wall clock on a loaded machine, or for a test service priced per invocation, would prefer a different schedule from one paying for CPU — and the units can disagree by more than the effect being measured (Section 7.2), which is why both are reported for every configuration. Separately, a candidate surviving the whole pool is *accepted*, and the pool is finite: passing a finite suite is weaker than being correct [21]. That does not affect the cost claims, which hold the acceptance criterion fixed across configurations, but it bounds what the repair-rate numbers mean.

The sandbox clock shapes the corpus silently. A wall-clock kill is applied to the reference program first, so a test the correct program cannot finish leaves the pool, and a task whose pool empties leaves the corpus. This keeps the oracle sound, but it removes faults non-randomly, from the expensive end — the end where the cost term would matter most.

8.4 Conclusion Validity

Sample size and skew. Ninety-seven tasks with three seeds is small for an effect whose per-task distribution is heavily right-skewed and dominated by a few expensive tasks: the top third by cost spread carries 79 % of the corpus's validation CPU. Comparisons therefore use the Wilcoxon signed-rank test on per-task totals rather than a mean-based test, every point estimate is reported with an interval, and the contrast where the aggregate and the per-task counts disagree is reported both ways (Table 6.7).

Clustering. The three seeds of one task share the task, the pool and the cost structure, so treating them as independent would understate every interval by roughly $\sqrt{3}$. Intervals come from a cluster bootstrap that resamples whole tasks.

Multiple comparisons. Four contrasts are declared in advance and carry a Holm correction. The ablation ladder, the cost-spread split and the coverage-check replay are reported with intervals and labelled exploratory, and no conclusion rests on one of them alone.

A significant p and an interval covering zero are not a contradiction. For `sched` against `frtp` the Wilcoxon test is significant ($p = 0.026$ after correction) while the bootstrap interval on total CPU covers zero. They measure different things: the test asks which configuration is cheaper on a *typical* task, and the answer is FRTP, on 45 tasks against 18; the interval asks about the *total*, which `sched` wins because its few wins are large. Both are reported, because reporting either alone would be misleading.

Chapter 9

Conclusion and Future Work

9.1 Conclusion

An LLM-based repair loop produces useful information on every iteration and then discards almost all of it. Every validation reveals which test rejected which candidate and what that test cost; every generation reveals which repair idea has already been tried. Current systems keep neither in a form the next iteration can use: schedulers rank tests by a count of past rejections and ignore how long a test takes, and prompts either carry nothing or carry a transcript that grows without bound.

This report described CASTER, which keeps that information in one small record per test case — how often the test has run, how often it rejected a candidate, and how much CPU those runs took — and spends it twice. **Cost-aware scheduling** ranks tests by an estimated rejection rate divided by an estimated cost, so an unpromising patch is rejected as cheaply as possible rather than merely in as few steps as possible. **Selected hints** replace the accumulated transcript with a small, freshly chosen set of concrete requirements, so the prompt stays a constant size while its content tracks what the session has just learned. The two share one record by construction, which is what makes them a system rather than two features.

The whole mechanism is three numbers per test case, updated in constant time. Nothing is trained, no profiling pass is needed, and neither half adds a program execution or a model call.

What the pilot showed. A pilot of 97 ConDefects Python tasks, three seeds each, twenty rounds per session, measured the scheduling half.

- Scheduling removed **60.8** % of validation CPU against the order the benchmark ships, interval $[-73.3, -36.3]$, while repairing exactly the same bugs on exactly the same rounds.
- Almost all of that came from **having a memory at all**. Any configuration that remembered something landed within a few percent of the others, and all of them roughly 60 % below the incumbent.
- The new part of the rule — **dividing by cost** — was 2.9 % cheaper than FRTP over a whole session, interval $[-11.0, +4.9]$, and that single figure hides the shape of the result. A rejection count is informative after one observation; a cost estimate

is a mean. Over rounds 1–5 the scheduler is 2.1 % *more* expensive for dividing by a number it does not yet know; over rounds 11–15 it is 10.6 % cheaper; the running total crosses over at round 10 and is still widening at round 20. The win also concentrates in the third of tasks whose test costs actually differ. It pays 13 % more executions for 14 % cheaper executions — the trade it is designed to make, and one that is invisible to a study reported in executions alone.

The generation half has not been run. The report is therefore a measured result for the scheduler and a declared protocol for the hints.

9.2 Future Work

1. **Run the full corpus, and run it for longer than twenty rounds.** The pilot screens; the corpus measures. Two of its open questions — whether the cost term beats FRTP, and whether a rate beats a count — have point estimates in the right direction and intervals that straddle zero, which is what a corpus-scale run exists to resolve. The budget matters as much as the corpus size here: the scheduler only overtakes FRTP at round 10 and the gap is still growing at round 20, so a twenty-round budget measures the cost term during the half of the session in which it is still learning. A longer budget is the cheaper of the two experiments and should come first.

2. **Run Block B.** The generation half is implemented and unit-tested and has not been measured. It is the half the duplicate-rate literature says is worth the most: Chen et al. [7] measured 65 % of a conversational repair system's patches to be duplicates.

3. **Add a benchmark with method-level test suites, such as Defects4J [23].** Every test in ConDefects is a whole-program execution, so nothing can be selected away and prioritisation is the only lever. A project-scale Java benchmark has a call graph and therefore has selection, which is where Lou et al. [22] find selection to dominate prioritisation. Two questions follow: does a cost-aware schedule still help once the cheap tests have been selected away, and is the right design a scheduler layered on a selector? That setting also tests the method's own failure condition directly, because method-level suites are more uniform in cost than contest pools.

4. **Add a second proposer model.** Everything the store learns is a statistic over the candidates one model produced. If different models make different mistakes, different tests are discriminating for each, and the scheduler re-learns per model — which is fine, since it learns online, but it would mean the reported effect size is one model's. If instead the same tests are discriminating across models, the rejection structure is a property of the *task*, which is a stronger claim: a rejection table could then be computed once per benchmark and reused, which is also what item 1 needs.

References

[1] Yonghao Wu, Zheng Li, Jie M. Zhang, and Yong Liu. ConDefects: A complementary dataset to address the data leakage concern for LLM-based fault localization and program repair. In *Companion Proceedings of the 32nd ACM International Conference on the Foundations of Software Engineering (FSE)*, pages 642–646, 2024. doi: 10.1145/3663529.3663815.

[2] Yuhua Qi, Xiaoguang Mao, and Yan Lei. Efficient automated program repair through fault-recorded testing prioritization. In *Proceedings of the 29th IEEE International Conference on Software Maintenance (ICSM)*, pages 180–189, Eindhoven, The Netherlands, 2013. doi: 10.1109/ICSM.2013.29.

[3] Claire Le Goues, Michael Pradel, and Abhik Roychoudhury. Automated program repair. *Communications of the ACM*, 62(12):56–65, 2019. doi: 10.1145/3318162.

[4] Zhihao Lin, Junhua Zhu, Mingyi Zhou, Xin Wang, Zhensu Sun, Renyu Yang, David Lo, and Li Li. To run or not to run: Analyzing the cost-effectiveness of code execution in LLM-based program repair. In *Proceedings of the ACM on Software Engineering (ISSTA)*, 2026. To appear. arXiv:2606.26978.

[5] Islem Bouzenia, Premkumar Devanbu, and Michael Pradel. RepairAgent: An autonomous, LLM-based agent for program repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2188–2200, Ottawa, ON, Canada, 2025. doi: 10.1109/ICSE55347.2025.00157.

[6] Chunqiu Steven Xia and Lingming Zhang. Automated program repair via conversation: Fixing 162 out of 337 bugs for $0.42 each using ChatGPT. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 819–831, Vienna, Austria, 2024. doi: 10.1145/3650212.3680323.

[7] Aolin Chen, Haojun Wu, Qi Xin, Steven P. Reiss, and Jifeng Xuan. Studying and understanding the effectiveness and failures of conversational LLM-based repair. *arXiv preprint arXiv:2503.15050*, 2025.

[8] Cuong Chi Le, Minh Le-Anh, Cuong Duc Van, and Tien N. Nguyen. Semantic evolution over populations for LLM-guided automated program repair. *arXiv preprint arXiv:2604.02134*, 2026.

[9] Lingchao Chen, Yicheng Ouyang, and Lingming Zhang. Fast and precise on-the-fly patch validation for all. In *Proceedings of the IEEE/ACM 43rd International Conference on Software Engineering (ICSE)*, pages 1123–1134, Madrid, Spain, 2021. doi: 10.1109/ICSE43902.2021.00104.

[10] Yuan-An Xiao, Chenyang Yang, Bo Wang, and Yingfei Xiong. Accelerating patch validation for program repair with interception-based execution scheduling. *IEEE Transactions on Software Engineering*, 50(3):618–635, 2024. doi: 10.1109/TSE.2024.3359969. arXiv:2305.03955.

[11] Naman Jain, Jaskirat Singh, Manish Shetty, Liang Zheng, Koushik Sen, and Ion Stoica. R2E-Gym: Procedural environments and hybrid verifiers for scaling open-weights SWE agents. In *Proceedings of the 39th Conference on Neural Information Processing Systems (NeurIPS)*, San Diego, CA, USA, 2025. arXiv:2504.07164.

[12] Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Demystifying LLM-based software engineering agents. *Proceedings of the ACM on Software Engineering*, 2(FSE):801–824, 2025. doi: 10.1145/3715754.

[13] Antonis Antoniades, Albert Örwall, Kexun Zhang, Yuxi Xie, Anirudh Goyal, and William Wang. SWE-Search: Enhancing software agents with monte carlo tree search and iterative refinement. In *Proceedings of the 13th International Conference on Learning Representations (ICLR)*, Singapore, 2025. arXiv:2410.20285.

[14] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36 (NeurIPS)*, pages 8634–8652, New Orleans, LA, USA, 2023.

[15] Sungmin Kang, Bei Chen, Shin Yoo, and Jian-Guang Lou. Explainable automated debugging via large language model-driven scientific debugging. *Empirical Software Engineering*, 30(2), 2025. doi: 10.1007/s10664-024-10594-x.

[16] Xin Yin, Chao Ni, Shaohua Wang, Zhenhao Li, Limin Zeng, and Xiaohu Yang. ThinkRepair: Self-directed automated program repair. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 1274–1286, Vienna, Austria, 2024. doi: 10.1145/3650212.3680359.

[17] Silin Chen, Shaoxin Lin, Yuling Shi, Heng Lian, Xiaodong Gu, Longfei Yun, Dong Chen, Lin Cao, Jiyang Liu, Nu Xia, and Qianxiang Wang. SWE-Exp: Experience-driven software issue resolution. *arXiv preprint arXiv:2507.23361*, 2025.

[18] Nikhil Parasaram, Huijie Yan, Boyu Yang, Zineb Flahy, Abriele Qudsi, Damian Ziaber, Earl T. Barr, and Sergey Mechtaev. The fact selection problem in LLM-based program repair. In *Proceedings of the IEEE/ACM 47th International Conference on Software Engineering (ICSE)*, pages 2574–2586, Ottawa, ON, Canada, 2025. doi: 10.1109/ICSE55347.2025.00162.

[19] Sebastian Elbaum, Alexey Malishevsky, and Gregg Rothermel. Incorporating varying test costs and fault severities into test case prioritization. In *Proceedings of*

the 23rd International Conference on Software Engineering (ICSE), pages 329–338, Toronto, ON, Canada, 2001. doi: 10.1109/ICSE.2001.919106.

[20] Gregg Rothermel, Roland H. Untch, Chengyun Chu, and Mary Jean Harrold. Prioritizing test cases for regression testing. *IEEE Transactions on Software Engineering*, 27(10):929–948, 2001. doi: 10.1109/32.962562.

[21] Zichao Qi, Fan Long, Sara Achour, and Martin Rinard. An analysis of patch plausibility and correctness for generate-and-validate patch generation systems. In *Proceedings of the 24th ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pages 24–36, 2015. doi: 10.1145/2771783.2771791.

[22] Yiling Lou, Jun Yang, Samuel Benton, Dan Hao, Lin Tan, Zhenpeng Chen, Lu Zhang, and Lingming Zhang. When automated program repair meets regression testing—an extensive study on two million patches. *ACM Transactions on Software Engineering and Methodology*, 33(7), 2024. doi: 10.1145/3672450.

[23] René Just, Darioush Jalali, and Michael D. Ernst. Defects4J: A database of existing faults to enable controlled testing studies for Java programs. In *Proceedings of the 23rd International Symposium on Software Testing and Analysis (ISSTA)*, pages 437–440, 2014. doi: 10.1145/2610384.2628055.

[24] Harold Jeffreys. An invariant form for the prior probability in estimation problems. *Proceedings of the Royal Society of London. Series A*, 186(1007):453–461, 1946. doi: 10.1098/rspa.1946.0056.